

Survey paper

A survey on Bluetooth Low Energy security and privacy[☆]Matthias Cäsar^{*}, Tobias Pawelke, Jan Steffan^{*}, Gabriel Terhorst

Fraunhofer Institute for Secure Information Technology SIT, Rheinstraße 75, 64295 Darmstadt, Germany

ARTICLE INFO

Keywords:

Bluetooth Low Energy
Embedded systems security
Mobile and wireless security
Privacy protections

ABSTRACT

Since its introduction in 2009, Bluetooth Low Energy (BLE) has become a remarkable success. Due to its unique properties of low power requirements and its ubiquitous availability in practically every smartphone, it outnumbered classic Bluetooth BR/EDR in most areas. It enabled a multitude of new product categories like smart watches or connected health care devices that would not be feasible without such a technology. We are currently facing version 5.2 of the specification which is the result of a number of major and minor revisions, each fixing problems of earlier versions while adding new features and capabilities. This includes more secure pairing methods, like secure connection pairing. Cyber security was considered from the beginning of the Bluetooth specification and has been improved with each specification release. On the other hand, security weaknesses in the specification as well as in individual Bluetooth stack implementations have been identified. Designing a secure BLE device, or analyzing its security is a complicated task due to the overwhelming number of possible configurations. As the specification introduces constantly new features and subtle changes regarding privacy and security, this will become an even more complex task. To the best of our knowledge, no systematic overview over the current state of BLE does exist that covers the security and privacy properties of the different BLE versions and features including known weaknesses and attacks in a single place. With this survey we want to fill this gap.

1. Introduction

Bluetooth is a set of short range wireless protocols operating in the 2.4 GHz industrial, scientific and medical (ISM) band. The Bluetooth specification is split into two major parts. One part is called *Bluetooth BR/EDR*, also known as *Bluetooth classic*, the other one is called *Bluetooth Low Energy (BLE)* which was added in version 4.0. Both are nearly completely independent protocols. Our focus here is on core BLE. We do not look into specialized BLE based parts of the protocol, such as Bluetooth Mesh, LE isochronous channels and LE audio.

As its name suggests, BLE is intended for low power devices, or to put in another way, devices with limited power supply (e.g. battery powered) as well as limited computational and memory capabilities. Since these devices should last as long as possible without replacing the battery, special communication protocols, like BLE are necessary.

Applications can be found in all areas, including ones with high requirements on safety and security such as electronic locks, alarm systems, process monitoring, or medical devices. These devices are often controlled and monitored via BLE by smartphones or laptops.

Being wireless, a BLE interface is particularly exposed to potential attacks. An attacker does not need physical access to the device and

has a low risk of being caught in action. This makes security of BLE interfaces a major concern. Potential attack goals include sniffing, denial-of-service (DoS), spoofing, injection of messages, partial or full takeover of a connection, tracking, and localization. The Bluetooth specification offers security measures against most of these threats, introducing multiple device pairing schemes, optional encryption and authentication of connections, or address randomization. However, those security measures are only effective if properly specified, used, and implemented correctly. Several security measures defined in early versions are flawed and serious weaknesses have been found that implementers need to be aware of.

Weaknesses within earlier versions of the specification are still relevant. BLE modules, SoCs and devices commonly remain in the market and in service for several years without providing an upgrade option. So even for newly designed devices it is often desirable to remain backwards compatibility with older versions of the protocol.

Document structure. We have divided our discussion of these aspects into multiple parts. After giving a brief and general overview of the BLE architecture in Section 2, we summarize and illustrate all relevant

[☆] This research work has been funded by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts within their joint support of the National Research Center for Applied Cybersecurity.

^{*} Corresponding authors.

E-mail addresses: matthias.caesar@sit.fraunhofer.de (M. Cäsar), jan.steffan@sit.fraunhofer.de (J. Steffan).

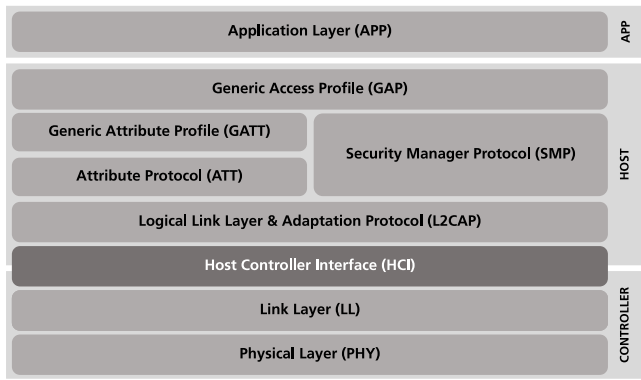


Fig. 1. Bluetooth stack.

changes introduced with each update of the specification in Section 3. In this way, we present the evolution of the security and privacy properties defined in the BLE specification.

In Section 4 we take a look into security and privacy issues and describe attack vectors like sniffing, jamming and tracking devices. Subsequently we summarize exploitable vulnerabilities and weaknesses that were identified throughout the evolution of the specification and implementations. Furthermore, we explain requirements and impacts of attacks and reference recommendations for mitigation.

We conclude our work with a short summary of aspects that need to be considered when designing a secure and privacy friendly BLE device in Section 5. However, this is no survey of publications on proposed countermeasures, as these often suggest improvements to the specification itself which are out of reach for device developers.

Throughout the text we made an effort to use appropriate language following the transition made by the Bluetooth SIG with version 5.3 of the specification. Mainly, the terms “master” and “slave” are being replaced with “central” and “peripheral”. This also affects many abbreviations derived from these terms. A full mapping between previous and new terms can be found in the SIG’s official mapping table [1].

2. General architecture

The Bluetooth stack shown in Fig. 1 can be divided into two major parts: *controller* and *host*. The former consists of the lower layers, which handle time-critical tasks, whereas the latter contains layers performing the high-level, more complex tasks. The communication between those two parts is done through the host controller interface (HCI). This way the host can control the lower layers through a set of commands and the controller can send data to the upper layers. The application interfaces with the most upper layers of the host part of the BLE stack.

2.1. Physical layer

The physical layer (PHY) of BLE defines the specific capabilities of the BLE radio and therefore properties of the analog circuit itself. BLE operates in the ISM band from 2.402 GHz to 2.480 GHz and therefore in the same frequency range as Bluetooth classic, Wi-Fi and many other popular wireless communication protocols. In BLE this band is divided into 40 radio frequency (RF) channels with 2 MHz spacing. Of these 40 channels, three are designated as primary *advertising channels* used for initial advertising and connection establishment. The remaining channels are general purpose channels used for the majority of communication. In order to avoid interference with other Bluetooth or RF traffic in the same band, BLE uses frequency hopping. During an established connection the physical channel is changed every few packets. Additionally, the three primary advertising channels are equally distributed over the ISM band to avoid disruption of advertising caused by interferences.

2.2. Link layer

The link layer (LL) handles low-level tasks like reception, processing and forwarding of protocol data units (PDUs). To achieve a responsive RF communication through the physical layer, all time-critical tasks are also done by the LL. This includes calculation and verification of cyclic redundancy check (CRC) values, security related functions like encryption and decryption of PDUs, as well as calculation and verification of the message integrity code (MIC). The LL also maintains a unique device identifier, the *Bluetooth device address* (BD_ADDR) which allows to distinguish different communication partners.

To manage the different connection states the LL implements a state-machine. As of Bluetooth version 5.2 there are the following seven LL states: *standby*, *advertising*, *scanning*, *initiating*, *connection*, *synchronization*, and *isochronous broadcasting*.

BLE allows *connection oriented* and *connection-less* communication. The following example illustrates the sequence of LL states during connection establishment between two devices (*A*, *B*) in a connection oriented scenario. The *standby state* is the default state, where the devices do not send or receive data. As soon as device *A* switches into the *advertising state*, it becomes an *advertiser*, which is usually performed by peripheral devices like smart watches or fitness trackers. In this state it will send advertising packets on the three advertising channels and listens for connection requests in fixed intervals. Device *B*, the scanner, switches into the *scanning state* and listens on the three advertising channels for advertisement packets, which is usually performed by central devices like smartphones or PCs. Upon reception of an undirected or directed advertisement packet, *B* goes to the *initiating state* and responds to the received advertising packet of device *A* with a connection request. Device *B* switches into *connection state* and device *A* follows suit as soon it received and processed *B*’s connection request. If *B* sends any PDU the connection is acknowledged and thereafter considered as established. Upon connection establishment, both devices act in different roles. As *B* was the initiator of the connection it adopts the *central* role and device *A* as prior advertiser the *peripheral* role. The central device controls the timings of subsequent transmissions during the connection.

In the alternative connection-less scenario, the advertiser sends only non-connectable and non-scannable advertisement packets, thereby becoming a *broadcaster* [2, 1343]. Nearby devices cannot establish a connection with a broadcaster, but can listen for information sent over the non-connectable advertisement packets.

2.3. Host controller interface

Communication between controller and host is defined by a set of host controller interface (HCI) commands and events. Implementations of the HCI may differ heavily, based on the use case. In larger devices like personal computers or smartphones the controller part is often implemented in a dedicated IC or module. The host part conversely, is implemented in software, running on the main CPU. HCI data in this case is exchanged via a hardware interface like SPI, UART or USB. In smaller low-power devices on the other hand, host and controller are often integrated in a single IC, sometimes even integrated with the main CPU or micro controller, also known as system on chip (SoC).

2.4. Logical link control and adaptation layer protocol

Regarding BLE, the main function of the logical link control and adaptation protocol (L2CAP) layer is protocol multiplexing. In general, L2CAP has several other features which are mostly used by Bluetooth BR/EDR and hence are not discussed here. Protocol multiplexing is necessary, since there is more than one upper layer protocol defined by BLE. To correctly route packets addressed to these upper layer protocols, packets are tagged with a protocol-specific *channel identifier* (CID).

2.5. Security manager

The security manager (SM) layer handles all parts of Bluetooth regarding security. It can be divided into a cryptographic toolbox and methods for both pairing and key exchange. The former contains all cryptographic functions for hash calculations and the generation of all keys used by Bluetooth. The latter was heavily reworked with Bluetooth version 4.2. For more detail on the specific pairing methods see Sections 3.1.2 and 3.3.2.

2.6. Attribute protocol

The attribute protocol (ATT) is a client-server protocol, which allows a device called *server* to expose attributes and their associated values to another device, the *client*. In Bluetooth LE both central and peripheral of a given LL connection can each be both an ATT server and client at the same time. Clients can discover, read and write attributes offered by the server. The server itself can indicate and notify attributes and their values to the client. Each attribute consists of a value and three properties associated with it:

- attribute type
- attribute handle
- access permissions

The attribute type is given by a universally unique identifier (UUID). These UUIDs are defined either by the Bluetooth SIG or directly by the manufacturer. The attribute handle is a 16-bit number, which allows the client to specify the attribute in requests. The access permissions of an attribute describe in which way the associated value can be accessed, e.g. if reading or writing is allowed. Access restrictions like encryption, authentication or authorization cannot be queried directly by the ATT.

On top of ATT the generic attribute profile (GATT) defines a data abstraction model for attributes. Single attributes are referred to as *characteristics*. A group of characteristics is called a *service* and multiple services are grouped into a *profile*.

2.7. Generic access profile

The purpose of the generic access profile (GAP) is to describe profile roles and define modes and procedures for the discoverability, connection and security of BLE devices. By utilizing features provided by the other layers of the stack, the GAP guarantees interoperability between devices of different manufactures. For BLE devices GAP classifies four profile roles: *broadcaster*, *observer*, *peripheral*, and *central*. The GAP defines which functionalities of other layers are mandatory or optional.

3. Bluetooth LE versions and features

The security properties of a BLE connection are defined primarily through the selected security mode, security level and the used pairing method. Privacy can be improved through the use of features like address randomization which have been introduced and tweaked over the various versions of the specification.

In the following sections we look into these aspects as they have developed over the history of the BLE protocol, beginning with version 4.0.

3.1. Version 4.0

Bluetooth version 4.0 marks the introduction of the Bluetooth LE protocol in 2009. It is now deprecated and to be withdrawn in 2022 as stated by the Bluetooth SIG [3]. The security and privacy mechanisms we describe in this section represent the initial state of BLE security. The following explanations also lay the basis for understanding changes introduced in later updates.

Table 1

Overview of BLE security modes.

Mode,		Pairing security	Message security	Since version
Level				
1,	1	No pairing	No security	≥ 4.0
	2	Unauthenticated legacy	Encryption + MAC	≥ 4.0
	3	Authenticated legacy	Encryption + MAC	≥ 4.0
	4	Authenticated LE-SC	Encryption + MAC	≥ 4.2
2,	1	unauthenticated	GATT layer data signing	≥ 4.0
	2	Authenticated	GATT layer data signing	≥ 4.0
3,	1	No pairing	No security	≥ 5.2
	2	Unauthenticated	Encryption + MAC	≥ 5.2
	3	Authenticated	Encryption + MAC	≥ 5.2

3.1.1. LE security modes

BLE is intended to support a wide range of application scenarios which can have quite different security requirements and constraints such as device capabilities or user interfaces. To meet those requirements, BLE is very flexible in the way its security mechanisms can be selected and combined. The options range from no security at all, to man-in-the-middle (MITM) protected key exchange with encrypted and authenticated communication. The supported sets of security features are organized into *security modes* and *levels*. Table 1 gives an overview of the security modes and levels introduced in Bluetooth specification 4.0 and later.

Bluetooth 4.0 distinguishes two security modes, subdivided into several security levels. In security mode 1 the connection is encrypted and authenticated using AES128-CCM in all levels but level 1, which provides no security. In security mode 2 the connection remains unencrypted, but data is signed on a higher layer of the Bluetooth protocol using a special key in order to verify the authenticity of the data. Security mode 2 can be sufficient if transmitted data needs to be trustworthy, but not necessarily confidential, such as the outside temperature measured by a sensor of an air conditioning system. Security levels 2 and above differ in how the pairing is performed. Pairing is a procedure which is initiated the first time a connection is established between two BLE devices in order to establish and distribute key material used for performing encryption, message authentication or data signing. The Bluetooth specification defines association methods which are distinguished between authenticated and unauthenticated pairing methods. During authenticated pairing, additional information is exchanged through an out-of-band channel or user interaction, so it cannot be eavesdropped or manipulated by an attacker with access to the wireless channel. This shall prevent the attacker from performing a MITM attack, since she does not have knowledge of the additional information. Unauthenticated pairing methods skip MITM protection in favor of a simplified user interface and experience.

3.1.2. Pairing procedure

The pairing procedure is performed to establish and distribute keys between two BLE devices in order to encrypt future connections or to sign the exchanged data. Therefore, this procedure is mandatory for every security configuration, except for security mode 1, level 1 (“no security mode”). Pairing consists of two mandatory phases shown in Fig. 2 and an optional third phase called bonding which is explained later. In phase 1 both devices exchange pairing features, like their input/output capabilities or if MITM protection or bonding is requested. The MITM flag must be set, when authenticated pairing is required according to the selected security level. The Bluetooth specification in version 4.0 defines two methods for authenticated pairing, “Passkey Entry” and “Out of Band”.

The actual key establishment is performed in phase 2. Depending on the I/O capabilities of the participating devices, the “Passkey Entry” procedure is slightly different. If one device has a display capability and the other device has an input capability, the device with the display

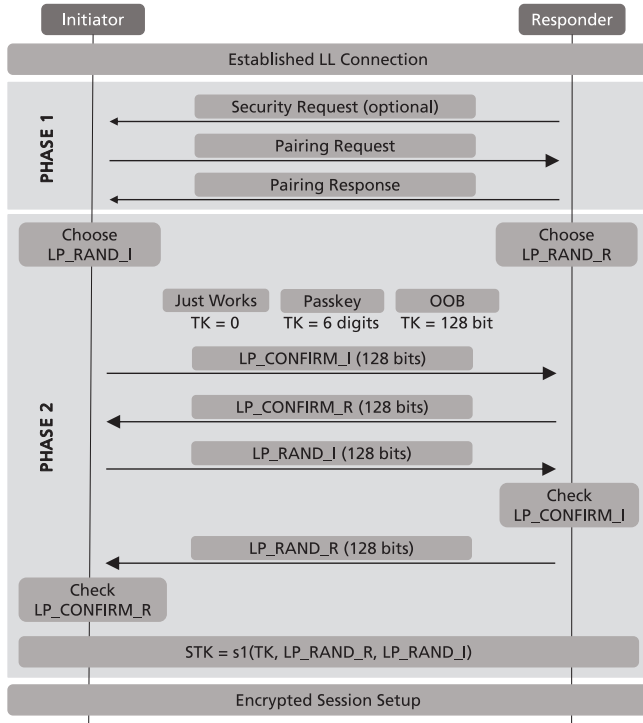


Fig. 2. Bluetooth pairing procedure phase 1 and 2.

generates a random 6 digit numeric passkey ('000000' to '999999'). This needs to be entered by the user on the keyboard of the other device. If both devices have "Keyboard Only" capabilities, the user must ensure that the keys entered on both devices are the same and as random as possible. The numeric passkey, which is at most 20 bits, is then zero padded to a 128 bit value and used as a temporary key TK . If the association is performed out of band, TK is a 128 bit value exchanged over a second transmission channel, like near field communication (NFC). For unauthenticated pairing *Just Works* is used where TK is simply set to "0".

Additionally, each device independently generates a random value, called LP_RANDOM_I for the central (initiator) and LP_RANDOM_R for the peripheral device (responder). In a next step the actual authentication is performed. Therefore, both devices calculate the 128 bit confirmation values $LP_CONFIRM_I$ and $LP_CONFIRM_R$ using their own random value LP_RANDOM_I resp. LP_RANDOM_R , the exchanged TK , both device addresses and parameters exchanged during phase 1. The calculation of the confirmation values are as follows:

$$p1 = \underbrace{\text{pairing resp. cmd.}}_{56 \text{ bits}} \parallel \underbrace{\text{pairing req. cmd.}}_{56 \text{ bits}} \parallel \underbrace{rat}_{1 \text{ bit}} \parallel \underbrace{0 \dots 0}_{7 \text{ bit}} \parallel \underbrace{iat}_{1 \text{ bit}} \parallel \underbrace{0 \dots 0}_{7 \text{ bit}}$$

where rat is the responding device address type and iat the initiating device address type.

$$p2 = \underbrace{0 \dots 0}_{32 \text{ bits}} \parallel \underbrace{ia}_{48 \text{ bit}} \parallel \underbrace{ra}_{48 \text{ bit}}$$

where ra is the responding device address and ia the initiating device address.

$$\begin{aligned} LP_CONFIRM_R &= AES128(TK, \\ &AES128(TK, LP_RANDOM_R \oplus p1) \oplus p2) \\ LP_CONFIRM_I &= AES128(TK, \\ &AES128(TK, LP_RANDOM_I \oplus p1) \oplus p2) \end{aligned}$$

Both devices then send their confirmation value to each other. To check the confirmation value, the peer's random LP_RANDOM_I resp. LP_RANDOM_R used when calculating the confirmation must be known. Therefore, the central device sends its random LP_RANDOM_I to the peripheral device, which then can verify $LP_CONFIRM_I$. If the validation is successful the peripheral sends its random LP_RANDOM_R in return whereby the central device is able to validate $LP_CONFIRM_R$. In case of success, both devices use LP_RANDOM_I , LP_RANDOM_R and TK to derive the short term key (STK). Therefore, the most significant 64 bit of the 128 bit LP_RANDOM_I and the 128 bit LP_RANDOM_R are discarded and then both values are concatenated as follows:

$$r' = LP_RANDOM_R' \parallel LP_RANDOM_I'$$

The STK is now calculated using the AES128 algorithm with the 128 bit r' as plain text input and the 128 bit TK as the key. This key is subsequently used to encrypt the, so far, unprotected communication according to the procedure described in Section 3.1.4.

Unfortunately this pairing procedure was later found to be vulnerable against offline brute-force attacks as described in Section 4.5 in detail.

3.1.3. Bonding procedure

The pairing procedure would have to be initiated every time a Bluetooth link layer connection is established in order to encrypt the connection. However, it is possible to exchange a long term key (LTK) during pairing which is stored by both devices in order to skip the pairing on future reconnects. This optional step must be requested during phase 1 and is called *bonding*. It is performed after phase 2 of the pairing procedure using the encrypted Bluetooth connection based on the STK. During this step, it is also possible to optionally distribute an identity resolving key (IRK) for resolving private device addresses as explained in Section 3.1.5 and a connection signature resolving key (CSRK). The CSRK can be used in unencrypted BLE connections to send authenticated data on the ATT layer in an attribute protocol PDU which can be verified by the receiving device to ensure the data is actually sent from the supposed device [2, 1539]. After the keys are distributed, the encrypted Bluetooth connection based on the STK is terminated and immediately reestablished according to Section 3.1.4 using the now available LTK.

The keys shown in Fig. 3 are generated by the peripheral (responding) device and sent over the encrypted connection to the central device. However, the central can also provide keys to the peripheral in case the roles are reversed in future connections. According to the Bluetooth specification, there are two different ways to generate LTK, IRK and CSRK [2, p 1690]. The first one is to generate these keys as random values with no relationship between each other and to store them in a database using an independently generated 16 bit encrypted diversifier (EDIV) as an index. The other option is to use a key hierarchy where the identity resolving key (IRK) is derived from identity root (IR) and the LTK and CSRK are derived from encryption root (ER), which reduces the storage requirements on the distributing device, since only IR and ER must be stored. In this case EDIV and a 64 bit $Rand$ are calculated depending on the IR and ER and also distributed to the peer device. Upon reconnect, the central sends EDIV and $Rand$ to the peripheral which is then able to recover the LTK and CSRK.

However, generating keys using a key hierarchy is not recommended, since it results in weak keys as described in Section 4.5.

It is important to know, that the privacy of the device is only protected against peer devices, that are not part of the set to which the IRK has been given. [2, p. 1657].

If the pairing procedure (phase 2) is somehow compromised, the attacker is also in possession of the later on exchanged keys, like LTK or CSRK, which are used for securing future Bluetooth connections.

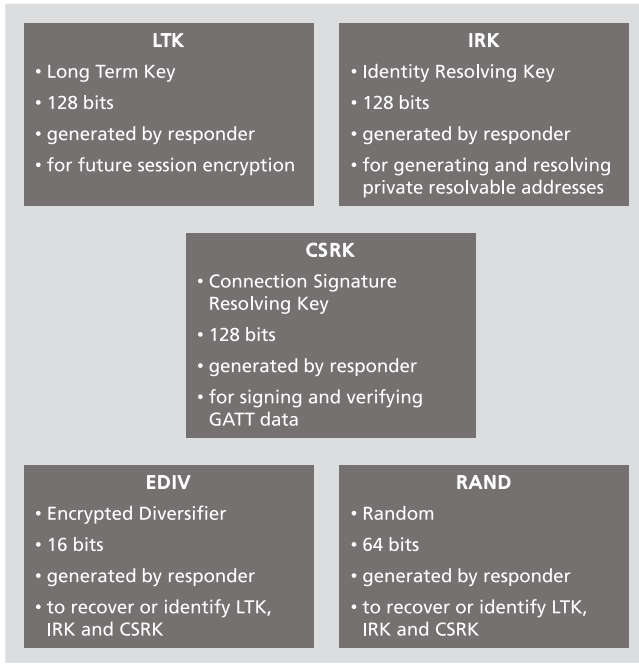


Fig. 3. Optional bonding procedure and additional key distribution phase.

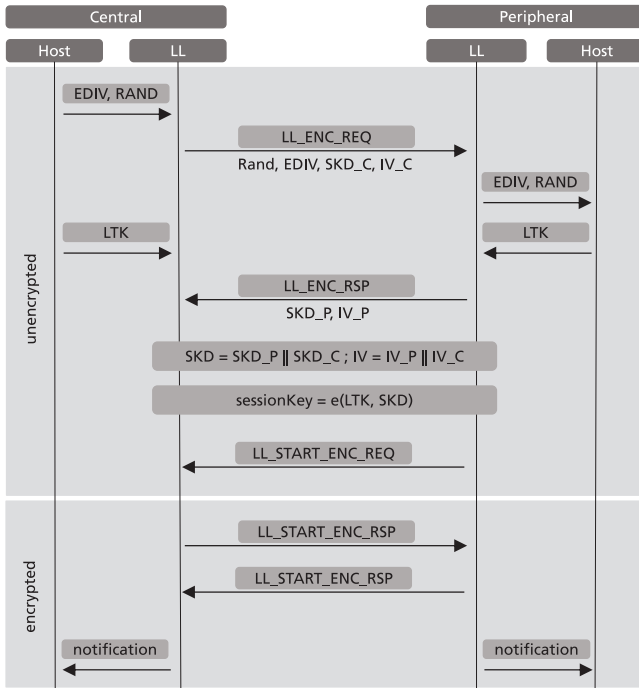


Fig. 4. BLE Encrypted Session Setup.

3.1.4. Session encryption

After pairing is successfully completed, the encryption key (STK or LTK) has been exchanged. This key can then be used to encrypt the BLE connection by performing the encrypted session setup procedure [2, p. 1660] shown in Fig. 4. If bonding was performed beforehand as described in Section 3.1.3, EDIV and Rand values were exchanged as a connection specific identifier for an LTK, respectively to recover the LTK. In this case, the central sends both values to the peripheral to allow the peripheral selecting the correct LTK from its database. If only pairing was performed, no EDIV and Rand values exist, since

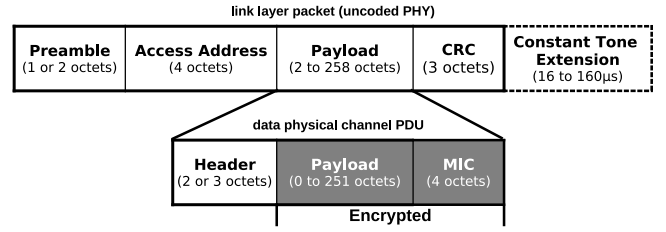


Fig. 5. BLE link layer packet format with encrypted parts.

the STK/LTK is not stored. In this case the corresponding PDU fields are set to zero. In addition to the two values, both devices generate and exchange a fresh part for the session key diversifiers SKD_C and SKD_P (128 bit random numbers) and the initialization vectors IV_C and IV_P (64 bit random numbers) using a link layer encryption request PDU (LL_ENC_REQ) [2, p. 3029]. Each device concatenates the random numbers as follows:

$$SKD = SKD_P || SKD_C$$

$$IV = IV_P || IV_C$$

In the next step a session key is calculated using the AES128 algorithm with the session key diversifier (SKD) as plain text input and the STK/LTK respectively as the key. The output is the session key, which is used for encrypting the new session. Finally, a three way handshake is performed indicating the start of the encrypted BLE connection using the AES-CCM algorithm [2, 3184], which shall be implemented according to IETF RFC 3610 [4]. In detail, the encryption is performed as follows. First, an authentication code is generated over the payload and the first octet of the header of the data physical channel PDU. According to the specification, this message authentication code is defined as message integrity code (MIC) to avoid confusion with the abbreviation of media access control (MAC). Furthermore, the first octet of the header is provided as additional authenticated data (AAD) to the AES-CCM algorithm, at which certain bits of the header are masked to zero. Then, the encryption is applied to the payload and the MIC. Fig. 5 shows the parts of a BLE packet which are covered by the encryption and the parts which are still transmitted in plain text. The information revealed from the remaining unencrypted parts of the packet is very limited. Preamble, access address (AA) and CRC do not contain sensitive information. The header of the data physical channel PDU contains, among others, an ID which is used to identify the packet type of the payload, two sequence number fields and a more-data flag. The ID indicates, whether the payload is a complete or fragmented L2CAP packet or a control PDU, which is used to exchange control messages on the link layer. The sequence numbers are used to implicitly acknowledge received packets and the more-data flag indicates continuation of sending data.

3.1.5. Privacy features

Devices that wirelessly transmit information that makes them recognizable are susceptible to tracking. A host of privacy issues can arise from the ability to associate and correlate an identifiable device with a person, a location, time, movement, and other nearby devices. For a BLE device the most obvious potentially trackable piece of information is the device address. This address is disclosed mainly during scanning, advertising, or initiating LL states. Recognizing this, the Bluetooth SIG introduced several address types to help disguising a device's identity summarized in Table 2. The Bluetooth specification requires that devices at least use a public or a random address.

Table 2
Bluetooth LE address types.

Address type	Lifespan	Random	Reconnectable
Identity addresses:			
Public address	Device lifetime		●
Static address	One or more power cycles	●	While unchanged
Private addresses:			
Resolvable private address	Timer based (15 min.)	●	●
Non-resolvable private address	Timer based (15 min.)	●	
Reconnection address (4.0 only)	Until next connection	●	Directed adv. only

Public address. For the public address the vendor needs to allocate an organizationally unique identifier (OUI) from the IEEE Registration Authority before production. Each address is composed of the OUI and a device unique part, mostly sequentially enumerated.¹ Public addresses are marked with the TxAdd flag set to 0 inside the PDU header.

As this address is unique for the complete lifecycle of the device, it is highly vulnerable to tracking throughout its entire lifetime.

Static address. Instead of using a permanent, unique address it is also possible to use a temporary, randomly generated *static address*. This can improve privacy, when renewed frequently. Besides that, there are no further requirements in order to support this address type. Static addresses can be identified by their most significant bits being set to 0b11. Once the address is changed a connection reestablishment is impossible as an advertiser would have no way to correlate saved encryption or authentication keys with the new address. The Bluetooth specification requires devices to keep their static addresses at least until the device is powered off and on again.

Private addresses. Devices implementing privacy support may use the already mentioned address types or the privacy specific ones, like the NRPA, RPA or *reconnection address*. Which address type is used in sent PDUs highly depends on the device's GAP role and whether privacy is enabled or supported. Claiming support for the privacy feature implies several implementation requirements for private address usage, which are discussed later in this section. The Bluetooth SIG recommends to regenerate a new private address every 15 minutes.

Like a static address the non-resolvable private address (NRPA) is also randomly generated. In contrast to the static address, it can be changed during runtime without the necessity of a power cycle. An NRPA can be identified by its most significant bits being set to 0b00. Like static addresses they also have the drawback that they cannot be used for reconnections.

A more sophisticated approach are the resolvable private address (RPA). They combine the advantage of a public and a private address, making it possible to prevent the leakage of a public address while maintaining the reconnection functionality. To generate an RPA a keyed-hash function similar to an HMAC is used. The necessary key, more precisely the identity resolving key (IRK) needs to be exchanged during bonding (Section 3.1.3).

An RPA consists of two parts, a keyed hash *hash* and a random number *prand*. To calculate the hash, the hash function *ah*, the IRK and some random number is required.

$$hash = ah(irk, prand)$$

The RPA is then calculated by the concatenation of the hash result and the random number.

$$RPA = hash || prand$$

A device wanting to reconnect to another device needs to scan for nearby addresses and try to resolve received addresses. This is

done by calculating the local hashes based on stored IRKs. If the local hash matches a hash part of a random address, the resolution was successful.

$$localHash = ah(irk, prand)$$

An RPA can be identified by its most significant bits being set to 0b10. The resolving of RPAs has to be done in the host, as the link layer is not capable of address resolution. Usage of RPAs therefore results in a higher power consumption due to necessary host wake-ups.

One important thing to be aware of is, that the privacy of a BLE device is only protected against peer devices, which have not received the IRK used for generating the resolvable private address [2, 1657].

The reconnection address is a special address type which is only used for directed advertising. A peripheral needs to support the privacy feature and provide the *Reconnection Address* characteristic in order to use this address type. Once a central connects to the peripheral, it can write a random reconnection address to the mentioned characteristic. In case the peripheral wants to send a directed advertising it shall use this address to protect the public identity of the central and its own.

Peripheral. A device in the *peripheral* role may use private addresses if it supports the privacy features. Furthermore, the usage of private addresses depends on the presence and value of the *Peripheral Privacy Flag* and the *Reconnection Address* characteristics on the peripheral. If the peripheral does not implement the peripheral privacy feature, it does not support privacy at all and needs to use a public or random static address. In case the peripheral provides the peripheral privacy flag and it is set to 0b1, the device may use a RPA as advertiser address for undirected advertising. If the reconnection address is also present and set, the peripheral may use its value as advertiser and initiator address for directed advertising.

Central. Central devices supporting the privacy feature shall use an NRPA during active scanning and a NRPA or RPA for connection establishment. If the *Reconnection Address* characteristic is present on the peripheral, the central should also write a new address into the characteristic. This allows the peripheral to use this address for directed advertisements. Bonded *central* devices without privacy support may disable the privacy feature upon connection establishment by setting the peripheral privacy flag to 0b0. This characteristic may only be writable as long as there is only one bonded device.

Usage of address types. The Bluetooth SIG is very lax on the enforcing of private addresses in case privacy is supported. Version 4.0 of the specification describes the privacy feature only for devices in the peripheral or central role. Further, the privacy feature is only specified for undirected and directed connectable and not for non-connectable or undirected scannable advertisements.

3.2. Version 4.1

Version 4.1 of 2013 mainly brought simplifications of the privacy features. It is now deprecated and will be withdrawn in 2023.

3.2.1. LE security modes and pairing procedure

With publication of the Bluetooth specification version 4.1 no changes were made regarding the LE security modes and the pairing procedure.

¹ The structure of a public address changed with revision 5.0 so an OUI is not required in all cases.

3.2.2. Privacy features

With the updated specification the privacy feature has been overall simplified. By removing the *Peripheral Privacy Flag* and the *Reconnection Address* characteristic, a central device cannot modify the privacy behavior of the peripheral anymore. The privacy section of the specification now also covers the non-connectable advertisement, but removed support for directed connectable advertisements. Furthermore, privacy profiles for the observer and broadcaster role were defined.

Peripheral. Devices sending non-connectable or scannable advertisements shall now use an NRPA or RPA. While using undirected connectable advertisements they shall use an RPA. The specification also states that peripherals should use undirected connectable advertisements for connection creation. When using the privacy feature, the specification recommends to exclude the device name in the advertising data.

Central. During connection establishment, a privacy-enabled device shall use the RPA as initiator address. In active scanning cases an NRPA or RPA shall be used.

Broadcaster and observer. Privacy-enabled devices in the broadcaster or observer role shall use an NRPA or RPA during advertising respectively scanning.

3.3. Version 4.2

Version 4.2 of 2014 marks a major update with a complete overhaul of the pairing mechanism in response to the vulnerabilities found in the previous versions 4.0 and 4.1. The newly introduced secure connections pairing was carried over to all of the following versions including the current one.

3.3.1. LE security modes

In Bluetooth specification 4.2 a fourth security level for security mode 1 is introduced. It is called “Authenticated LE Secure Connections pairing with encryption” and addresses security flaws of Bluetooth LE in specification 4.0 which are discussed in Section 3.1.2. Besides, the new security level is designed to use only FIPS-approved algorithms. Also, it should be noted that security level four does not replace the security levels 1 to 3, but is defined in addition to it as shown in Table 1. Accordingly, using no authentication and no encryption is still compliant with Bluetooth specification version 4.2. However, to prioritize a secure connection over backward compatibility, a secure connection only (SCO) mode [2, 1375] is defined additionally. Here, usage of security mode 1, level 4 is enforced. Otherwise, the devices will refuse to establish a connection. To meet the requirements of security mode 1, level 4 a new pairing procedure is defined in Bluetooth specification version 4.2 called LE Secure Connections pairing (LE-SC) which is discussed in Section 3.3.2. The old pairing procedure which meets the requirements of security mode 1, level 2 and 3 is now known as *legacy pairing* but without any further improvements regarding security.

3.3.2. LE secure connections pairing (LE-SC)

The sequence chart of the new LE-SC pairing procedure is shown in Fig. 6. One major improvement over legacy pairing is the use of elliptic curve Diffie-Hellman (ECDH) key exchange with the P-256 elliptic curve. LE-SC enables both parties to agree on a shared key without transmitting it over the air and hence, without the risk of it being eavesdropped.

In phase 1 both devices exchange pairing features, as already discussed in Section 3.1.2 with one difference compared to legacy pairing. To indicate the capability of performing pairing over LE secure connection procedure, BLE devices set the SC flag of the pairing request respectively the pairing response message [2, 1670]. However, it is important to note, that LE-SC will only be performed, if both devices

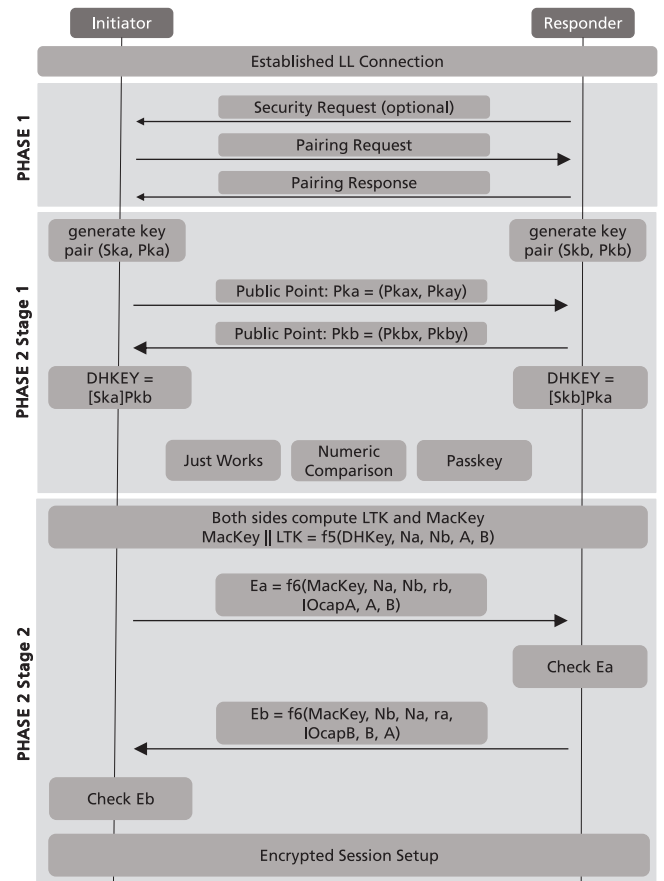


Fig. 6. Bluetooth LE secure connection pairing procedure.

have set the SC flag to one. If only one device supports LE-SC and thus sets the SC flag, legacy pairing is performed to maintain backward compatibility according to Bluetooth specification.

As shown in the sequence chart, the procedure is nearly symmetrical and both devices perform the same steps. (Some asymmetry may be introduced in through the authentication step.) Phase 2, stage 1 starts with each device picking a random private key value (Ska and Skb) to compute public keys (Pka and Pkb) which are then transferred to the peer device. Each device subsequently computes the same shared secret $DHKey$ from its private key value and the peer's public key. The next step differs slightly depending on the selected association model, which are described in detail later on. However, all association models result in having two random values Na , Nb on both sides, one generated by each device.

In stage 2, the LTK and MacKey are calculated by each device using $DHKey$, both device addresses A and B , and the output of the association step (Na and Nb). This procedure ties the Diffie-Hellman key exchange, the association procedure and the device addresses cryptographically together and thereby prevent attacks targeting parts of the pairing procedure.

Ultimately, confirmation values Ea and Eb are calculated by both sides from the MacKey, Na , Nb , both Bluetooth device addresses A , B , the capabilities of the devices ($IOcapA$, $IOcapB$), and the values ra , rb which depend on the used association model. Then the confirmation values are exchanged and verified. This step ensures that both devices have agreed on the same $DHKey$ without sending it over the air and to verify the genuineness of all exchanged parameters. If this step succeeds, both devices have established an LTK, which can then be used for encrypting the connection.

In contrast to the legacy pairing procedure (Section 3.1.2), LE-SC pairing does not generate an STK first and optionally an LTK if bonding

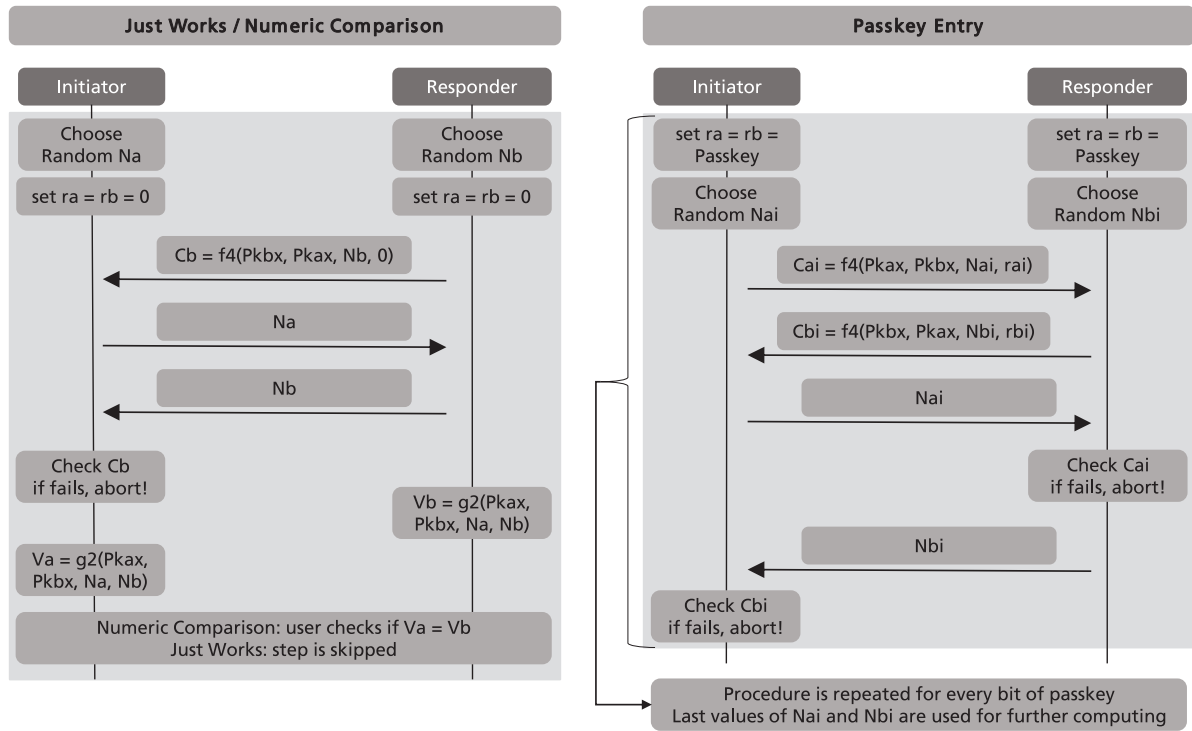


Fig. 7. BLE LE-SC association models: (A) Just Works/Numeric Comparison (B) Passkey Entry.

is requested. Instead, during LE-SC the LTK is directly calculated using ECDH. Depending on the bonding flag of the SMP pairing request message, the LTK is either stored for future connection establishments or it is dropped after the connection is terminated.

Since the LTK is calculated using the ECDH parameters, it is not possible to generate the key based on a key hierarchy as described Section 3.1.3. Therefore, the values EDIV and Rand are also no longer necessary and simply set to zero where indicated.

Additionally, if bonding is performed, IRK and CSRK can be generated and exchanged. As in previous versions of the specification, the IRK is used for resolving private device addresses according to Section 3.1.5 and the CSRK can be used to send authenticated data over an unencrypted connection between two devices.

In contrast to the LTK, IRK and CSRK can either be generated randomly or based on a key hierarchy as described in Section 3.1.3.

LE-SC association models. The LE secure connection pairing supports four different association models, *Just Works*, *Numeric Comparison*, *Passkey Entry* and out-of-band (OOB), of which *Just Works* does not provide any MITM protection. The procedure of *Just Works* and *Numeric Comparison* are almost identical, except for the last step, where the user compares numeric values on both connected devices. A detailed message sequence chart is shown in Fig. 7 (A). During *Passkey Entry* procedure, which is shown in Fig. 7 (B), the user inputs an identical passkey into both connected devices. There is also a slightly different version, where a passkey is generated and displayed on one device and then entered on the second device by the user. If Bluetooth devices are capable of transmitting information over an out-of-band channel, the Bluetooth specification allows an OOB authentication protocol which can be used either if OOB communication is possible in both directions or only in one direction.

Just works and numeric comparison. During this association procedure [2, p. 1647, 1700], both devices select a freshly generated random number N_a , N_b in order to prevent replay attacks. Then, the non-initiating device computes a hash value C_b containing both x -values of the Diffie-Hellman public keys Pk_{ax} , Pk_{bx} and the random number N_b

which is then sent to the initiating device. This step is a commitment to the random number and the x -value of the public key and prevents an attacker from changing these values in the following communication. After both devices have exchanged their random numbers N_a , N_b , the initiating device can verify the commitment C_b using N_b and abort the procedure on mismatch. A mismatch can only occur if the values Pk_{ax} , Pk_{bx} and N_b are changed during transmission by an attacker or a communication failure. However, an active MITM can easily perform two independent Diffie-Hellman exchanges, one with device A and another with device B, without the commitment check failing. Since the *Just Works* procedure is completed after this step, it is obvious, that MITM attacks cannot be detected.

Choosing *Numeric Comparison* instead, a further step is added to this procedure which mitigates the lack of authentication. During this step both devices generate a 6-digit confirmation value which is displayed on both devices, in order to be confirmed by the user on both devices. Therefore, the security level of this method highly depends on the user's awareness of the importance of this step. The confirmation value consists of both x -values of the Diffie-Hellman public keys Pk_{ax} , Pk_{bx} and both random numbers N_a , N_b . This ensures, that neither the public keys of ECDH nor the random values can be changed by an attacker without a deviation of the confirmation values.

Passkey entry. Authentication with *Passkey Entry* [2, p. 1649, 1704] can be performed in two different ways. Either, the user enters an identical passkey into both devices or one device generates and displays a passkey, which is to be entered on the second device by the user. Both ways have in common, that the mutual authentication relies on the uniqueness and randomness of the used passkey.

The following procedure is repeated for every single bit of the passkey. This means, for a 6-digit passkey which is 20 bits, the procedure is repeated 20 times. For every iteration both sides generate a 128 bits long nonce and commit on one bit of the passkey by computing and exchanging a commitment value containing both x -values of the Diffie-Hellman public keys Pk_{ax} and Pk_{bx} , the nonce N_{ai} or N_{bi} and the respective bit of the passkey rai or rbi . The nonce is added to the hash to prevent replay attacks as well as to impede brute forcing the bit

of the passkey. To bind the passkey procedure to the ECDH exchange performed beforehand, both x -values of the public keys are included in the hash. After the initiating device has sent the nonce Na_i , device B can verify the commitment hash Ca_i with its knowledge of the passkey bit and in case of success send the nonce Nb_i to device A, whereby device B is able to verify the hash Cb_i . Both devices immediately abort the procedure in the event of failure. If the procedure is successfully completed after 20 iterations, the last generated nonces Na_{20} and Nb_{20} are used for computing the LTK and MacKey as described in Fig. 6.

Security properties of passkey entry. One of the main security properties of the passkey entry association model is that it relies on a bit commitment scheme, where both participants commit on a shared secret (passkey) bit by bit while keeping it hidden to others. After both parties have committed on a particular bit of the passkey, it is mutually disclosed. A MITM must participate on this bit commitment procedure in order to convince both parties to be the legitimate communication partner. Therefore, the MITM must correctly guess the bits of the passkey one after another, since a bit is only disclosed when both parties have committed to it. Brute forcing this bit beforehand is impeded by the long nonce included in the hash containing the bit of the passkey. Committing on the wrong bit is detected during the disclosure phase and results in the other party immediately aborting the authentication procedure. According to the Bluetooth specification a specific passkey should only be used once, which prevents a MITM from using obtained information of the passkey for a new attempt. However, as mentioned above, there are two ways to perform this authentication procedure. If a passkey is entered by a user into both devices, it is up to the user to ensure that the same passkey is not reused (see also).

Furthermore, to prevent attackers from disassociating the ECDH exchange from the passkey authentication procedure, both x -values of the public keys are included in the commitment hash. Otherwise an attacker could perform the ECDH exchange independently from the passkey authentication. In this case, a possible attack scenario would be that a MITM performs the ECDH exchanges with both devices independently and during authentication procedure the messages are simply passed through between both devices. In this way the MITM would be able to successfully finish the BLE pairing procedure whereas both legitimate devices have no chance to detect the attack.

Out-of-band authentication. To perform the authentication out-of-band [2, p. 1651, 1708], a priori authentication information must be available and announced via the “OOB data flag” in the SMP Pairing Request message. Since this is the first message of the BLE pairing procedure, it is assumed, that the OOB authentication has already been performed before the BLE pairing starts. However, the Bluetooth specification is vague regarding the order of the messages exchanged during pairing with OOB authentication. There is no single exhaustive message sequence chart depicting all necessary messages from the very first to the very last. To the best of our knowledge, Fig. 8 shows the complete procedure of the Bluetooth LE secure connection pairing with OOB authentication. Furthermore, there are some different approaches to perform OOB pairing. One option is, that the pairing is triggered by the OOB interface which is covered by our sequence chart in Fig. 8. First both devices generate their Diffie–Hellman key pairs and select a random number ra or rb . The second step is to compute a commitment hash consisting of the x -value of the Diffie–Hellman public key $Pkax$ or $Pkby$ twice and the random number. Then, the OOB communication is performed depending on the capabilities of the devices. If both devices can receive and transmit OOB data, messages 1a and 1b are exchanged, otherwise only 1a or 1b. BD_ADDR denotes the Bluetooth device address of the device referred to.

After the OOB communication is finished, a Bluetooth LE connection between both devices is established and the pairing request and response messages are sent with the OOB data flag set whether OOB authentication data is present or not. If a device had received OOB data, in a next step the commitment hash is verified against the public

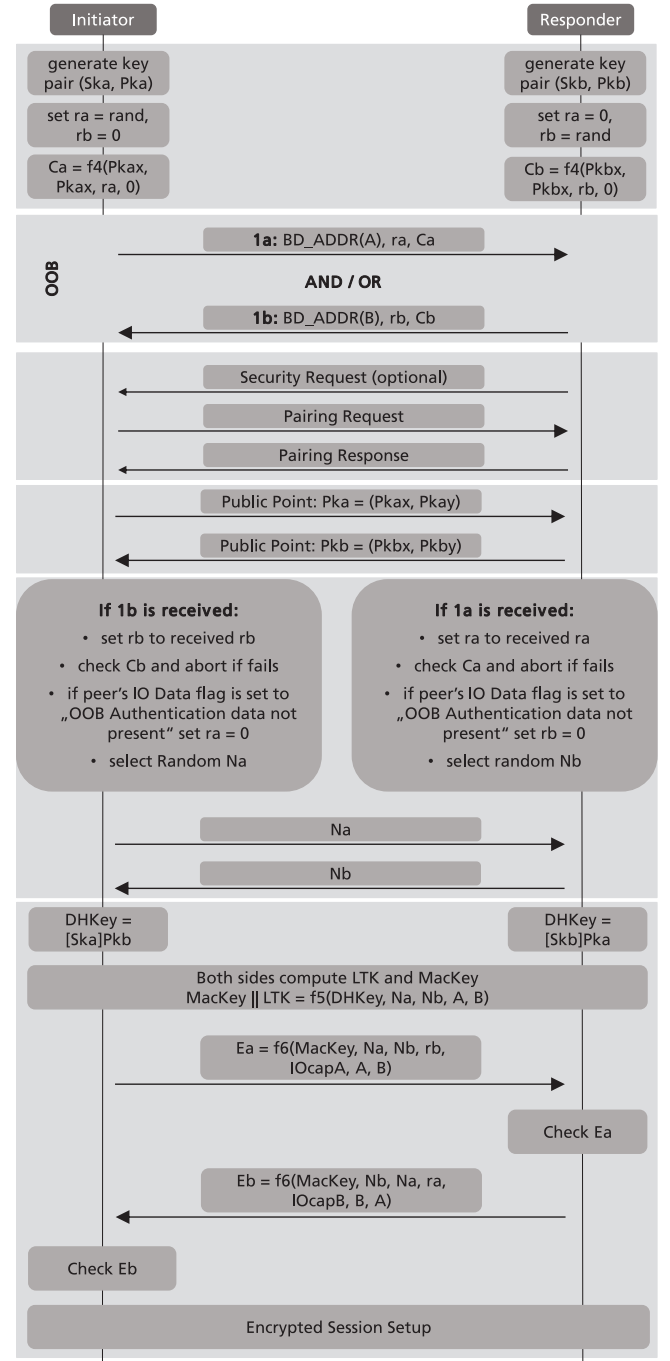


Fig. 8. BLE LE-SC authentication method OOB.

key exchanged in-band. If the verification fails, the pairing procedure is aborted. Next, both devices verify whether the peer had received the OOB information. The presence of the OOB information is announced by the peer device using the “OOB data flag” of the pairing request respectively in the pairing response. So, if the peer device had set the flag, the device can be sure that its OOB information was received. In case, one device detects that OOB information is not present on the peer device, the device sets its own r value to 0. This is done to correctly verify the confirmation values Ea , Eb in step 13. The further sequence of the pairing is equal to Fig. 6.

Bluetooth classic secure simple pairing. Even though this is a survey on Bluetooth LE, the similarity of LE-SC with Bluetooth classic BR/EDR

secure simple pairing (SSP) [2, 271, 983] should be noted. SSP obviously served as a blue print after BLE legacy pairing had to be replaced due to security issues. The relationship of both protocols raises the question if and how attacks and vulnerabilities of SSP do also affect LE-SC pairing. The so far described sequences of LE-SC pairing including the association models (Just Works, Numeric Comparison, Passkey Entry and OOB) are identical to the sequences of SSP, but there are differences regarding the used cryptographic algorithms and functions during pairing. In general, LE-SC relies on the AES-CMAC algorithm whereas SSP relies on HMAC-SHA-256. Additionally, the confirmation hash, which is generated at the end of the pairing procedure is calculated using the MacKey for LE-SC and the DHKey for SSP. A comprehensive list of all functions can be found in the Bluetooth specification in section “Cryptographic Toolbox” [2, 1625] for BLE pairing and “Cryptographic Function Definition” [2, 994] for SSP.

Ultimately, it needs to be verified on a case-by-case basis whether security issues found in one of the pairing protocols are transferable to the other pairing protocol. However, it is important to keep this possibility in mind.

Cross-transport key derivation. Additionally to the similarity of LE-SC and SSP, there is also a protocol defined by the Bluetooth specification to generate keys for both transports (LE and BR/EDR) without the necessity to pair twice. cross-transport key derivation (CTKD) [2, 1401, 1654] is available for devices supporting secure connections over BR/EDR and LE transports. For instance, Bluetooth headphones use the Bluetooth classic protocol in order to receive audio from a source like a smartphone, but they can also support additional features like configuring the headphones which is performed over a BLE connection. Without CTKD, users would have to pair the device twice for both transports. To increase the ease of use, the Bluetooth SIG defined CTKD, which allows to perform pairing using one transport and derive a second key for the other transport [2, 271, 1658]. However, CTKD increases the attack surface significantly, since on one hand, it increases the complexity of the security architecture, and on the other hand exploiting a potential vulnerability in one pairing procedure can also have an impact on the security of the other transport. Recent attacks described in Section 4.6 are showing that the actual implementation of CTKD is a real threat for the security of the Bluetooth protocol.

3.3.3. Privacy features

With version 4.2 several privacy related functions like the *Central Address Resolution* characteristic, controller-based RPA generation and resolution, and two privacy modes were included.

If a peripheral wants to send directed connectable advertisements with an RPA as initiator address, it should check if the *Central Address Resolution* characteristic is set. If the central address resolution is missing or unset the initiator address is allowed to contain the identity address, which is a public or random static one.

With the inclusion of “Link Layer Privacy” the address generation and resolution can now be done inside the controller. This allows BLE devices to decrease power consumption during advertising and scanning as the host does not need to be woken up.

Furthermore, the specification introduces two privacy modes: *device privacy* and *network privacy* mode, which is controlled on a per device basis. In device privacy mode a device is only concerned about its own privacy. It should accept advertising PDUs with their identity address as well as PDUs containing a peer’s private address regardless of a prior exchanged IRK. In network privacy mode a device is only allowed to accept PDUs containing private addresses as device address, which is the default mode. The device privacy mode is necessary e.g. if the legacy device only supports identity addresses as peer address.

3.4. Version 5.0

In version 5.0 of 2016 the main update with regard to security was the strengthening of security mode 1, level 4.

3.4.1. LE security modes

For security mode 1, level 4 (LE-SC), a key length of 128 bit is now mandatory. Prior to version 5.0, a key length in the range of 56 bits to 128 bits was allowed and could be negotiated between devices. This could result in connections using a key length below 128 which today is generally considered insufficient for secure symmetric cryptography [5–7].

Enforcing a 128-bit key length also prevents the KNOB downgrade attack discussed in Section 4.5.

3.4.2. Privacy features

The new specification limits the usage of the device privacy mode. Starting with BLE version 5.0 the peripheral can indicate that it only utilizes an RPA by setting the *Resolvable Private Address Only* characteristic to 0b0. If the characteristic absent, no assumptions about the address usage can be made. This way bonded central devices can configure their privacy mode accordingly. In conclusion, the laxer device privacy mode is only allowed in cases where the *Resolvable Private Address Only* characteristic is missing on the peripheral.

3.5. Versions 5.1 and 5.2

Versions 5.1 and 5.2 of 2019 mainly brought new features unrelated to security. There is one small change aiming to mitigate vulnerabilities with regard to cross-transport key derivation.

3.5.1. LE security modes

In Bluetooth version 5.1 no changes are made regarding LE security modes.

In specification version 5.2 “LE Isochronous Channels” are introduced which required the definition of an additional security mode 3 (see Table 1).

3.5.2. Privacy features

With publication of Bluetooth specification 5.1 and 5.2 no privacy related changes were made.

3.5.3. Other security related changes

With version 5.1 new restrictions were introduced with regard to cross-transport key derivation (CTKD) [2, section 14.1, p. 1401] which no longer allows an LTK to be overwritten with a key established through a weaker method.

4. Known weaknesses and attacks

While in the previous sections we explained all of the security and privacy mechanisms introduced into BLE over the course of its development, we now will look into weaknesses and vulnerabilities that were discovered in the same time-frame. First, we discuss inherent limitations of BLE’s security and privacy with regard to generic attacks like sniffing, jamming, or tracking. In Section 4.5 we then come to weaknesses discovered in the pairing protocols which are a corner stone of BLE security. In Section 4.6 we summarize known attacks against already paired devices which have established a shared secret. Finally, in Section 4.7 we look into generic weaknesses due to flaws commonly found in BLE implementations.

4.1. Sniffing BLE

In a sniffing attack a third party aims to obtain information communicated between BLE devices. This is often done through passive eavesdropping, but may also involve actively triggering the transmission

of desired data. A BLE sniffer is typically a combination of software and hardware. Solutions range from off-the-shelf BLE development boards with adapted firmware to dedicated sniffing hardware. Since the advertisements are easily captured by design, sniffing BLE usually refers to monitoring upcoming or established connections.

In order to sniff BLE communication, multiple obstacles have to be overcome. As mentioned in Section 2 the used ISM band is divided into 40 RF channels of equal width: 37 general purpose and three primary advertising channels. The initial connection setup is performed on one of the three advertising channels. During this setup phase multiple connection specific parameters are exchanged. For the purpose of sniffing, the three most relevant are the used hopping sequence, the access address (AA) and the hopping interval of the upcoming connection.

A connection itself consists of a sequence of connection events. A connection event describes the fixed length period of time during which the two communication partners exchange messages on one physical channel. Between communication events both devices simultaneously switch to the next channel according to the predefined hopping sequence.

Every single packet of both communication partners starts with a preamble and a connection specific AA. Both of those fields are essential to detect the start of a valid BLE packet within the constant data stream received and demodulated from each physical channel. Since these connection parameters are transmitted only once during connection initiation, an attacker either has to capture these during the setup phase or reverse engineer them when the connection is already in progress. For this reason it is substantially easier to sniff a just upcoming connections than one already in progress.

Currently available commercial BLE sniffers only support the first case, i.e. sniffing only connections that are established after the sniffer had started monitoring the primary advertising channels. However, there are multiple open-source projects which are able to monitor BLE connections, which are already in progress. One of those tools is the Ubertooth project. It was initially developed in 2011 as a Bluetooth classic test tool. In 2013 Mike Ryan built a BLE sniffer on top of the Ubertooth project [8]. His implementation allows for monitoring upcoming and already existing BLE connections.

In order to sniff connections already in progress, the before-mentioned critical parameters have to be recovered: AA, CRCInit, hopping interval, and hopping sequence. By design, all connection events contain at least one empty packet sent by the central device to the peripheral and vice versa. This leads to a vast amount of empty packets being present in every BLE connection. Ryan utilizes these empty packets in order to filter valid AAs on a single frequency. During the setup phase the central device assigns a connection specific initialization value for the CRC linear feedback shift register (LFSR). In order to verify the CRC for each packet this 24-bit seed has to be recovered. The CRC calculation done by using the LFSR is reversible though, which means with enough valid packets received the correct CRCInit value can be identified. By waiting on a single frequency and measuring the time between possible connection events on this frequency, the hop interval can be recovered [8, 4]. Lastly, the hop increment is recovered by measuring the arrival time of connection events on two different channels.

In 2018 Damien Cauquil published another open-source project capable of sniffing connections already in progress: BtleJack [9]. Cauquil also leverages the many empty packets being present in every BLE connection in order to restore the AA. In contrast to the Ubertooth project, BtleJack implements some additions, one being the possibility to catch up on later changes to the channel map of a connection.

Finally, it must be noted that BLE encryption only protects the contents of the payload of BLE packets. The preamble, AA, PDU header, and CRC fields are transmitted in plain-text. This means encryption has no effect whatsoever on the possibility of transmissions being observed by a sniffer in radio range. Even though an attacker cannot directly access the encrypted data, it is still possible to confirm close proximity to the target.

4.2. Man-in-the-middle attacks

Whereas sniffing refers to only receiving packets, a MITM attack describes an attacker being in the position to relay, drop or otherwise manipulate the connection. In case of BLE this attack requires two additional BLE devices to impersonate the two legitimate ones. Both targets, the peripheral and central devices will connect to the malicious devices controlled by the attacker. All messages then will first be exchanged with the malicious devices, who further relay the data to the legitimate ones.

Two open-source tools implementing MITM attacks on BLE are GATTacker and BTLEjuice. GATTacker was developed by Slawomir Jasek and presented at the Black Hat USA in 2016 [10]. Damien Cauquil published BTLEjuice in the same year at the DefCon 24 conference [11]. In essence, both tools implement similar functionality and work by the same principle. They allow to impersonate the target peripheral by serving the same GATT characteristics and services. By changing the BD_ADDR of the malicious peripheral device, the advertising packets will be sent identical to those of the legitimate peripheral. To maximize the success rate it is important for the target central to connect only to the malicious and not to the legitimate peripheral. To this end, both tools suppress the legitimate advertising packets by keeping the target peripheral connected to their central device, whilst the malicious peripheral sends packets at the shortest interval possible. Once both connections are established all packets sent and received are relayed via the malicious devices. This effectively allows the attacker to manipulate the connection by dropping, forwarding or changing specific packets as desired.

As already described in 3.1.1, BLE provides multiple security mode and level combinations to prohibit MITM attacks. Contrary to what its name suggests, setting the MITM flag during phase 1 of the pairing will *not* lead to an authenticated pairing method automatically. If one of the two devices has insufficient I/O capabilities, the MITM flag is ignored. Only by configuring the correct *Attribute Permissions* on the GATT server MITM protection can be enforced.

4.3. Jamming, session hijacking and packet injection

Jamming describes the deliberate blocking, and therefore suppression of specific parts of a communication or the target medium as a whole.

Operating in the open and heavily occupied ISM band at 2.4 GHz, Bluetooth is designed from the ground up to cope with RF interference. Its main strategy to avoid collisions with transmission of other devices or protocols is the use of a variable channel hopping sequence. However, deliberate jamming may cause havoc beyond unintended collisions. Consequently, the possibility of jamming should be considered explicitly during the development of BLE products. In case of BLE, jamming can either target multiple channels or specific packets or connections. For instance by jamming all three primary advertising channels an attacker can theoretically suppress connection establishments and advertising packets.

Further, shown in Section 4.1 it is possible to reconstruct key parameters of an already running BLE connection. This allows for targeting specific packets in single connection events and suppressing these. During the connection setup a supervision timeout value is set by the central device. As soon as one device of the connection did not receive a single packet for this prescribed amount of time, the connection is considered lost. By jamming only packets sent by the peripheral to the central device, an attacker can trigger the timeout in the central device. Since the timeout was triggered only in the central device the attacker can step in as central device and hijack the BLE connection. This attack was published in 2018 by Damien Cauquil and implemented in the tool BtleJack [9].

In 2021 R. Cayre et al. published “InjectaBLE” [12] which demonstrates the possibility to inject a single packet into an ongoing connection. A connection in BLE consists of a chain of connection events,

each starting with a packet sent by the central to the peripheral. Due to a feature called *Window Widening*, the peripheral starts listening for this packet a specific amount of time before it actually is expected to be sent. This allows for a race condition in which an attacker can inject a single packet into an ongoing connection. By utilizing this injection in conjunction with different packets sent each time R. Cayre et al. state the following attack scenarios: Illegitimately using a device functionality, hijacking the central, hijacking the peripheral or both of them simultaneously. The last attack scenario leads to a full MITM attack, which, in contrast to the previously discussed ones, can be performed on an ongoing connection.

4.4. Privacy related weaknesses

Even though product vendors implement Bluetooth LE's privacy feature, their devices may leak personal data from the user. Publications revealed privacy flaws in several products despite implementing privacy capabilities.

Device tracking. One major privacy concern is location-based user tracking. When using public or long-living static random device address this is a quite easy task [13,14]. That is the reason why the Bluetooth SIG tries to address this issues with the introduction of the privacy feature allowing the device to change its address frequently. In theory this would make long term tracking much harder. However, many manufacturers seem to use uniquely identifiable data in their advertising messages jeopardizing the effectiveness of address shuffling. Even though some of the product vendors shuffle those identifying tokens at some point in time, they do not synchronize it with the address randomization. This allows adversaries to correlate address changes with the unique parts inside the advertisement message [15–17]. Since advertisements are sent periodically in most cases, an adversary can perform this attack completely passively and likely undetected.

In 2019 Celosia et al. [18] showed how to leverage GATT based information to improve user tracking. For this an adversary needs to establish a connection to the subject. Upon connection establishment available services and characteristics are queried, including their value, handle, UUID and access properties to calculate a fingerprint. In many cases this information can be accessed without any further confirmation by the subject, making this an attractive information source. This fingerprint can often be mapped to a small group of devices or even to single devices as their work shows. To increase the difficulty for finding unique features, the authors recommend restricting access to services and their characteristics to authenticated devices only. As the characteristics *Device Name*, *Appearance* and *Service Changed* need to stay unprotected, they should only contain generic information and no unique information. Furthermore, they recommend to limit the list of discoverable characteristics and services to authenticated devices as well. This is contrary to the Bluetooth specification which says that this list is not considered private or confidential.

Nonetheless, a device might still be trackable as the privacy concept using the IRK to generate resolvable private addresses protects only from devices which are not in possession of the underlying IRK. Furthermore, the requirements for the generation and refresh of the IRK are so lax, device makers are even allowed to leverage a fixed IRK, assigned once during manufacturing [2, 1657]. This implies, a device owner has to ensure not to connect to untrustworthy devices to prevent that the IRK does fall into the wrong hands. In conclusion this means in the worst case, a once distributed IRK means lifetime traceability for this device.

Inferring user activity from traffic analysis. For many personal devices the number, frequency, length, or type of advertised BLE messages correlates with the user's behavior. For instance, for fitness trackers it was found that depending on the movement speed of a person, a frequency change of advertised messages can be observed [14]. Another more recent publication showed that it is possible to distinguish various actions of smartphone users, like using certain apps or setting pages, screen locking and unlocking or even answering calls from the advertised data [19].

Inferring user location by mobile apps. Device tracking can also occur on the user's device itself, especially where the user can install third party software. Advertising companies are always searching for new possibilities to improve their fingerprinting mechanism and fight against the emerging restrictions to identifiable device information. BLE scanning apps can leverage advertising data or BLE device addresses to infer a device's location and possible social environment.

With this information it is possible to generate fingerprints, allowing to link gathered information about the user across sandbox borders through nearby BLE devices [20]. Of particular interest are Bluetooth Mesh [21] and BLE beacons like Eddystone or iBeacons for location inferring, due to being mostly installed in a fixed location. Hence, both major mobile OS vendors require the user to explicitly grant location permissions, by today. Apps granted the permission to scan for nearby devices can still use this information to generate fingerprints. Ultimately, the user does not know what location related information the app processes only locally and which might also get shared with third parties [22].

4.5. Weaknesses in pairing protocols

Over the years, multiple weaknesses were discovered and published in BLE pairing and key establishment procedures. Some of the attacks summarized below are applicable to both Bluetooth LE and Bluetooth classic (BR/EDR). In accordance with the scope of this work and to avoid confusion we will only discuss aspects relevant for BLE. However, it should be noted that a BLE device that supports BT classic too, may be vulnerable due to its BR/EDR interface and attacks may even affect BLE security (see e.g. the BLURtooth attacks described in Section 4.6). An overview of the attacks described in this and the following section can be found in Table 3.

Many of the attacks described below assume a strong attacker who is able to sniff, suppress, inject, and manipulate messages at will during the pairing phase. This capability was formalized by Dolev and Yao [23] in 1983 for the purpose of protocol analysis and is now generally referred to as "Dolev–Yao adversary model". With a wireless protocol like BLE, an attacker in principle has full medium access, and therefore Dolev–Yao capabilities could be assumed. In practice however, as we discussed in Sections 4.1–4.3, aspects like frequency hopping or required knowledge of the AA complicate sniffing and injection.

Offline brute-force attack on legacy pairing. The pairing method of the initial BLE specification versions 4.0 and 4.1, which is now called *legacy pairing*, described in Section 3.1.2, was found to be vulnerable against offline brute-force attacks [8]. This is due to the fact that an attacker who is able to eavesdrop the wireless communication during the pairing procedure only needs to find the correct passkey (having only six decimal digits of entropy) through an exhaustive search in order to derive the negotiated cryptographic key. In legacy just-works pairing mode not even this is necessary, as the passkey is fixed to "0" here. Both pairing modes therefore have to be considered completely broken and the STK and LTK compromised if it can be assumed that an attacker can eavesdrop the communication during the pairing phase. Due to the simplicity of the attack, the legacy passkey mode provides no practical security benefit over the just-works mode. On the contrary, it may provide a false sense of security.

A variant of the attack in which the attacker actively participates in the pairing process without knowing the passkey beforehand was published in [24] in the same year.

The only remaining pairing option for BLE version 4.0 and 4.1 that is not affected by these attacks is OOB mode. In OOB mode the implementation of a secure key exchange procedure is "left as an exercise" to the developer of the BLE devices. In practice, OOB mode is rarely used, partially due to limited support by popular smartphone platforms. However, BLE 4.0 and 4.1 legacy pairing still provides a security benefit

Table 3

Summary of attacks during and after the pairing process (● = affected, ○ = affected in special cases).

Attack	Pairing	Affected versions						Additional precondition
		4.0	4.1	4.2	5.0	5.1	5.2	
Attacks against pairing process:								
Offline brute force	legacy	●	●					
Fixed coordinate invalid curve	LE-SC			●	●			
KNOB	LE-SC	●	●	●	○ ^a	○	○	
	Legacy	●	●	●		●	●	
Pairing method confusion	LE-SC			●	●	●	●	PE + NC support
Passkey reuse	LE-SC			●	●	●	●	Implementation flaw
BlueMirror	Both ^b	●	●	●	●	●	●	PE can be enforced
Attacks against already paired devices:								
BLURtooth CTKD key overwrite	LE-SC			●	●	○ ^c	○	BT classic + LE
BLESA spoofing	both	●	●	●	●	●	●	Implementation flaw
SCO mode downgrade	LE-SC			●	●	●	●	Implementation flaw

^aMitigated only for security mode 1, level 4.^bBlueMirror Attack has low impact with legacy pairing.^cCountermeasures introduced with 5.1 are only effective in some cases.

over security mode 1, level 1 because the described attack requires the attacker to eavesdrop the pairing phase or to participate in it. It therefore reduces the window of opportunity significantly compared to using no encryption at all.

Use of key hierarchy results in weak keys. As mentioned in Section 3.1.3, during legacy pairing the LTK, CSRK and IRK can be generated with one of two methods: as independently generated and stored *random numbers*, or derived through a *key hierarchy* from fixed values and a randomized 16-bit diversifier. The key hierarchy method requires less resources, but may reduce the keys' entropy to only 16 bit. LE-SC pairing instead directly results in an LTK. However, the CSRK and IRK can still be generated randomly or through the key hierarchy as in legacy pairing (see Section 3.3.2).

As noted by Claverie and Lopes-Esteves [25, Section 5.1] in 2020, use of the key hierarchy method reduces the available key space to only 2^{16} possibilities for a given device. If a device is paired often, this may result in the same LTK being generated twice. A key collision is certain to be observed within $2^{16} + 1$ pairings. Due to the birthday paradox there is a 50 % chance of a key collision to be observed during 302 pairings. Based on this, the authors propose a testing strategy to assess whether a given device uses the key hierarchy or randomization for key generation.

An attacker may exploit this weakness by establishing multiple pairings with the victim device until all possible LTKs are recovered. The attacker can then decrypt all past and future connections of the devices and spoof both end-points of the connection using the compromised LTK.

This issue is mentioned in the Bluetooth Core Specification since version 4.0 [2, p. 1692].

Fixed coordinate invalid curve attack on LE-SC. A weakness in the improved LE Secure Connections pairing (LE-SC) introduced with version 4.2 (see Section 3.3.2) was first published in 2018 as "Fixed Coordinate Invalid Curve Attack" (CVE-2018-5383) by Biham and Neumann [26] and acknowledged by the Bluetooth SIG in its first official security notice [27]. The attack also applies to *secure simple pairing* (SSP) of Bluetooth BR/EDR from which LE-SC was derived and affects versions 2.1 to 5.0.

LE-SC is based on an ECDH key exchange. In phase 2, stage 1 of the pairing procedure (see Fig. 6) the devices exchange EC public keys $Pka = (Pkax, Pkay)$ and $Pkb = (Pkbx, Pkby)$. However, the specification does not require that the receiver verifies that the received public key actually lies on the prearranged elliptic curve. Indeed, transmission of the y -coordinates would not be necessary in the first place as they could be calculated from the x -coordinates and the curve. Moreover, during

the authentication phase (see Fig. 7) the check values Ca and Cb are calculated by function $f4$ only from the x -coordinates $Pkax$ and $Pkbx$ of the public keys. The y -coordinates are not authenticated through the passkey or numeric comparison.

This weakness allows an attacker who is able to exchange the y -coordinates $Pkay$ and $Pkby$ during transmission with own chosen values – in this case 0 – to force the public keys on both sides to degenerate values from a small subgroup, and ultimately to recover the LTK (Dolev-Yao-model [23]). The attack is only successful in 25% of the cases if the attacker only manipulates the public key messages, and with a probability of 50% if the attacker is in a MITM position during the whole pairing process. In the rest of the cases, the pairing will appear to have failed for at least one of the Bluetooth devices.

Cremers et al. [28] later showed that the attack is also possible against only one of the devices if the other device is not vulnerable. A method to test whether a device is vulnerable is proposed in [25]. In the same publication it is also noted that a device that does not verify a received public key and does not change its private key between pairing attempts, may leak information to an attacker sufficient to reconstruct its private key.

As countermeasure, the Bluetooth SIG added the requirement to check that a received public key lies on the curve to the specification in erratum document 10734 [29].

Key negotiation of bluetooth (KNOB) attack on legacy pairing and LE-SC. The "Key negotiation of Bluetooth" (KNOB) attack was published in 2019 [30,31] (CVE-2019-9506). It mainly affects Bluetooth BR/EDR, but is applicable to both Bluetooth LE legacy pairing and secure connections (LE-SC), too — however, with a reduced window of attack and less severe impact [31,32, part VI].

The vulnerability was found in the feature exchange phase (Fig. 6, phase 1) of BLE pairing. Here, the devices negotiate a maximum entropy for the LTK of 7 to 16 bytes. The integrity of this exchange is not verified. An attacker who is able to manipulate or exchange these messages can downgrade the LTK's entropy to 7 bytes (56 bits).² As the session key (SK) used to secure future connections between the devices is derived from the LTK, its effective entropy is limited to 7 bytes, too. The assumed attacker capabilities correspond with the Dolev-Yao-model [23].

Whether a device accepts a low LTK entropy depends on its particular implementation. The authors of [32] found all of 19 evaluated

² For Bluetooth BR/EDR, the key negotiation is performed and can be attacked at each reconnection of two already paired devices. The key entropy can be downgraded to only 1 byte (8 bits).

BLE devices to be vulnerable. Moreover, a low entropy was found to be accepted in security mode 1, level 4 which, according to the specification should guarantee an entropy of 16 bytes (128 bits).

The Bluetooth SIG responded to the KNOB attacks in its second security notice [33] and in erratum 11838 [34] in which it only makes changes with regard to Bluetooth BR/EDR: enforcement of 128-bit entropy in BR/EDR security mode 4, level 4 despite a potentially negotiated lower entropy, and a minimum entropy of 7 bytes (56 bits) for all other modes becomes mandatory. For BLE, enforcement of 128 bit entropy for security mode 1, level 4 is mandatory starting with version 5.0 of the specification. Compliant BLE 5.0 devices therefore are not affected by the KNOB attack if they use LE-SC with mode 1, level 4. However, 5.0 and later devices may still be vulnerable for instance if they fall back to legacy mode when pairing with an older device. Falling back to legacy pairing could be prevented by using the secure connection only (SCO) mode.

Pairing method confusion attack on LE-SC. The “Pairing method confusion attack” (CVE-2020-10134) was discovered in 2020 [35] and affects the security of Bluetooth LE secure connections (LE-SC) pairing except OOB mode up to the, at that point, current version 5.2.

The attack exploits a design flaw in the association model negotiation phase in which the devices choose one of the modes *out-of-band* (OOB), *just-works* (JW), *numeric comparison* (NC), or *passkey-entry* (PE) based on their security requirements and I/O capabilities (availability of display or keyboard). The exchanged capabilities are not authenticated. This allows an attacker in a MITM position to perform a pairing with each of the user's devices while using different device capabilities and thus association models. Specifically, the attacker forces one side to perform NC, the other side to perform PE with the same numeric value. As a result, the user is displayed a six-digit number on the NC side and asked to enter a six-digit passkey into the other device. In a user study conducted by the authors, none of the participants was able to distinguish the attack from a legitimate end-to-end PE pairing. If the user enters the NC value as a passkey, the attacker can complete the pairing process with both devices.

Obviously, the attack only works with devices supporting PE and NC mode respectively. However, JW mode lacks MITM protection in the first place and therefore can be directly attacked by an attacker in a MITM position. Devices that use PE with a fixed passkey (e.g. printed on the case) in violation of the Bluetooth specification, can be attacked in other ways as discussed in the next paragraph below.

The Bluetooth SIG responded to the vulnerability with a security notice [36] in which it recommends that vendors make NC and PE mode clearly distinguishable for users by using language that warns them not to enter NC numbers as passkeys.

LE-SC Pairing with reused passkey. Depending on the devices' I/O capabilities, the LE secure connections (and Bluetooth BR/EDR SSP) *passkey entry* (PE) pairing mode can be executed in one of two ways: 1. The passkey is either selected by one of the devices and displayed in order to be entered into the other device by the user. 2. The user chooses a passkey and enters it into both devices. In order to authenticate the peer device, the passkey is then verified bit by bit through a commitment protocol (see Fig. 7). The security of this protocol relies on the assumption that only the peer device has knowledge of the correct passkey. However, during the commitment scheme one bit of the passkey is disclosed in each step. An attacker can obtain all bits of the passkey by eavesdropping the full commitment protocol. It therefore is inevitable that the same, now disclosed, passkey is not used a second time. Otherwise a MITM attack becomes possible for an attacker who has obtained the passkey. This attack was first described for Bluetooth BR/EDR SSP [37–39], but is valid for LE-SC passkey entry pairing, too [40].

A reuse of the same passkey is very likely if the user has to select and enter it on both devices. An attacker may actively provoke a second attempt of performing PE pairing by disrupting the pairing protocol

e.g. through selective jamming after obtaining the full passkey by eavesdropping the commitment protocol. Most users will likely not get suspicious if the pairing fails at the first try and assume that they made a mistake while entering the passkey in one of the devices. If the user retries pairing with the same passkey, a MITM attack becomes possible.

Passkey reuse may also occur in the second mode where the passkey is provided by one of the devices. Some vendors use a fixed passkey for devices with limited capabilities. For instance, the passkey may be set at manufacturing time and printed onto the device's case.

BlueMirror Reflection attacks. “BlueMirror” is a collection of seven attacks published in May 2021 [41], of which three affect BLE pairing, three affect Bluetooth Mesh provisioning, and one affects BR/EDR legacy pairing. We will only discuss the attacks affecting BLE here. The core idea of the attacks is *reflection* in which an attacker with Dolev–Yao capabilities [23] transmits unmodified messages back to their original sender during a symmetric authentication protocol.

In case of the BLE legacy pairing (attack “BLE-A” in the original paper), reflection during the commitment step results in identical confirmation values on the initiator's side. As a result, authentication succeeds. However, the attacker cannot recover the secret key. Therefore, this attack has no immediate impact, but it may enable further attacks that require the initiator to be in the authenticated state. For the attack to succeed, the initiator needs to accept messages from a peer sharing its own BD_ADDR as this is part of the confirmation value.

The second attack against BLE is called “PE-A1” and affects LE-SC. It works in a similar way as BLE-A, but requires that the attacker can force the initiator to use passkey entry by sending corresponding I/O capabilities. The attacker then reflects the public key exchange and each message of the commitment scheme back to the initiator. As with BLE-A this results in a successful completion of the authentication step without disclosing the key to the attacker. However, the attacker is able to recover the passkey as it is disclosed bit by bit with each commitment message. The knowledge of the passkey can be used in a third attack (called “PE-A2”) to establish an authenticated connection with the legitimate responder. The attack does not succeed if one of the peer devices uses OOB mode or if both devices offer DisplayYesNo I/O capabilities.

The Bluetooth SIG responded to attack BLE-A in [42] and acknowledged the combination attack PE-A1+PE-A2 in a security notice [43]. As a counter measure, the SIG recommends to reject public keys received from a peer that are identical to the own public key value.

4.6. Attacks against paired devices

The attacks summarized in the previous section require an attacker to observe the pairing process or to interfere with it. This narrows the attack window significantly. In the following we will discuss attacks that do not have this restriction and can be applied against already paired devices.

BLURtooth CTKD Key overwrite attack. The “BLURtooth” or “BLUR” attacks (CVE-2020-15802) were published in 2020 [44]. They exploit vulnerabilities in the cross-transport key derivation (CTKD) feature introduced with Bluetooth version 4.2. CTKD is a convenience feature intended for devices that support both Bluetooth BR/EDR and BLE. It allows devices to derive a BLE LTK from a BR/EDR link key (LK) and vice versa. In this way, only a single pairing process is required in order to allow secure connections for both BR/EDR and BLE transport modes. In a test of various laptops, smartphones and headsets, the authors of BLURtooth found CTKD to be widely supported by Bluetooth stacks implemented in hardware and software.

As BR/EDR and BLE pairings become interchangeable, the attack surface is greatly increased. The authors describe multiple attacks exploiting five identified weaknesses that can be used alone or in combination:

Central impersonation over BLE: It is assumed that the victim devices *A* and *B* have established a secure BR/EDR connection and support BLE and CTKD. The attacker impersonates *A* by spoofing its address and sends a just-works pairing request to *B* over BLE. If successful, *B* negotiates a BLE LTK with the attacker and derives a new BR/EDR LK from it which replaces the LK negotiated with *A*. The attacker thus takes over *A*'s role and the original connection to *A* is interrupted.

Peripheral impersonation over BR/EDR: peripheral impersonation is only possible over BR/EDR as it does not clearly distinguish the central and peripheral roles. If the secure session between *A* and *B* is interrupted, e.g. due to the central impersonation attack described before, the attacker can send a just-works pairing request to *A* via BR/EDR. As above, if the request is accepted by the victim *A*, a new BR/EDR LK and BLE LTK are derived between *A* and the attacker, replacing the original BR/EDR LK of *A* and *B*.

MITM attack: By combining central and peripheral impersonation, the attacker gets into a MITM position.

Unintended sessions attack: If a device mainly uses BR/EDR, but also supports BLE and CTKD, an attacker may achieve a BLE pairing with this device even if it is actively connected. The attacker gains both a BLE and BR/EDR connection and can then try to exploit vulnerabilities over both protocols.

For these attacks to work, besides supporting BR/EDR, BLE and CTKD victim devices must accept the attacker's pairing requests. Nevertheless, in a test of 13 devices with ten different Bluetooth chips supporting Bluetooth versions 4.1 (with back-ported CTKD) to 5.1, the authors of BLURtooth found all of them to be vulnerable [44].

The Bluetooth SIG acknowledged the weaknesses in a Security Notice in September 2020 [45] in which it recommends to use the CTKD restrictions "mandated in version 5.1 of the Bluetooth Core Specification versions 5.1 and later." This likely refers to new restrictions introduced in [2, section 14.1, p. 1401] which no longer allow an LTK to be overwritten with a key established through a weaker method by the other transport via CTKD. For instance, an LTK negotiated with security mode 1, level 4 may not be replaced with one derived through an unauthenticated association method like just-works. However, this does not prevent CTKD attacks if the victim devices were paired with just-works in the first place. The "Unintended sessions attack" should also not be affected by the new restrictions.

BLESA spoofing attacks. The "BLE Spoofing Attacks" (BLESA) were identified in 2019 and published in 2020 [46]. BLESA enables spoofing of the GATT server side during reconnection of previously paired BLE devices if the implementation of the client is flawed.

The server can require different security levels for accesses to individual attributes. Read or write access to a sensitive attribute may require authentication and encryption on link-layer established during the previous pairing phase. Another attribute such as the device name may be readable without link-layer security. The authors identified several weaknesses resulting from this flexibility: The security level for an attribute request can be established in a reactive or proactive way.

In the *reactive* case, the server receives an access request for an attribute requiring authentication and encryption over a plain-text connection. It responds with an error messages to tell the client to enable link layer security. An attacker who is able to spoof the server can accept the client's plain-text request instead and send a response containing false attribute data to the client.

In *proactive* mode, the client tries to enable link layer encryption and authentication before sending the attribute request. If the server is spoofed, enabling link layer security fails and the client is supposed to terminate the connection. However, the authors found that this was not the case in some implementations, including Android and Apple iOS (CVE-2020-9770) which continued the connection in plain-text mode.

4.7. Implementation flaws and platform specific weaknesses

In the previous section we summarized security weaknesses due to design flaws or gaps in the Bluetooth specification. In the following we give a short overview over security relevant bugs that were found in individual Bluetooth stack implementations. These typically affect only a single series of Bluetooth chips, or a single vendor's stack or OS. While weaknesses in Bluetooth stacks of operating systems (OSs) can be fixed with the regular software update process, this is not so easy with stacks implemented in hardware modules and SoCs.

Downgrade attacks on BLE 4.2 secure connection only mode. In 2019 and 2020 Zhang et al. published their extensive research [47,48] on the security of secure connection only (SCO) mode introduced with BLE 4.2 (see Section 3.3), which is intended to enforce security mode 1, level 4. The authors note that, according to the Bluetooth specification, SCO mode forces the GATT server (typically the peripheral device) to authenticate the GATT client (typically the central device) using secure association methods (authenticated LE-SC pairing). However, the Bluetooth specification does not require the GATT client to authenticate the GATT server. The authors showed that under certain conditions this flaw can be exploited for downgrade attacks in various ways. For instance, the central device acting as GATT client may unknowingly connect to a spoofed peripheral device acting as GATT server that does not enforce SCO mode, resulting in plain text communication. The authors found that Android, iOS, macOS, Windows 10, and Linux were vulnerable for these attacks. In Android, iOS and macOS there is also no way for apps to enforce or verify a certain secure association mode, to respond to resulting errors, or to remove a suspicious device bonding. In total they describe eight types of attacks against central and peripheral devices that partially build on top of each other or rely on further implementation flaws such as improper access control for characteristics. Attack impacts include injection of false data, information leakage, DoS, and MITM.

Countermeasures would ideally be implemented on the level of the affected BLE stacks or SDKs. The authors implemented a prototype solution for Android 8.

Co-located app attacks. Many BLE devices are accompanied by a dedicated smartphone app that provides a device specific UI and guides users through device discovery and setup. BLE pairing and bonding however does not happen on app level, but on OS level. The connection to the BLE device and the LTK is managed by the mobile OS, e.g. Android or iOS. In [49] it was shown that Android OS does not restrict access to a bonded or connected device to the official app. Instead, any installed app with Bluetooth and location access privileges is allowed to request the OS to scan for and connect to an already bonded device. Moreover, if a connection to the BLE device is currently already established, any app with just Bluetooth permission can send GATT requests to the device in parallel to the original app. This happens without indication to the user.

Currently the only option to prevent device access by unauthorized third-party apps is to implement an application layer authentication and access control mechanism on top of BLE.

BlueDoor. The BlueDoor attack [50] published in 2020 combines four weaknesses in the BLE protocol respectively in particular implementations, which results in the possibility to obtain personal information from many BLE consumer devices such as fitness trackers.

The first weakness is, that the BLE protocol relies on device addresses, which can easily be spoofed to pretend to be a legitimate device. The second weakness is, that most central devices, analyzed by the authors, do not enforce encryption when the peripheral device rejects the encryption request or when the encryption response from the peripheral is not received. This can be achieved by either jamming the response or by injecting the reject packet. The third weakness is, that a central device can mimic a low capability device and thus force

pairing with the lowest security, i.e. Just Works. This is possible, since the pairing method with the highest security level is selected according to the capabilities of both devices. If the central device pretends to have no capabilities, Just Works pairing is performed. The fourth weakness is that according to the authors most devices implement the authentication decoupled from the encryption on the application layer. This leads to the situation that even when the encryption is disabled (e.g. by exploiting the second weakness), the authentication procedure still works. Since the connection is unencrypted an attacker could simply forward the authentication packets and bypass the process.

By combining those four weaknesses, the authors present that they can convince BLE devices to grant access to GATT attributes with “encrypted and authenticated” access permissions.

It is not clear to us, how this could be achieved for GATT layer access permissions with an application layer authentication mechanism.

Sweyntooth. In 2020 Garbelini et al. published the “Sweyntooth” collection consisting of 17 different vulnerabilities [51]. Several chip and software vendors are affected by the flaws. As many product makers use the same chips, the number of affected end devices is even higher. The authors estimate more than 480 vulnerable devices.

Eight of the vulnerabilities result in a device restart, five in a DoS and two allow to bypass security measures. As many of these vulnerabilities are due to buffer overflows, they might be advanced to gain remote code execution (RCE). To trigger most of the flaws no pairing nor bonding is necessary.

Other implementation flaws. The “BlueBorne” vulnerability collection published in 2017 [52] is noteworthy as it seems to be the first systematic security analysis of widespread Bluetooth stack implementations (e.g. Windows, Linux, Android). The identified vulnerabilities include RCE and logic flaws but only affect BR/EDR. “BrakTooth” [53] from 2021 is a collection of similar vulnerabilities affecting the BR/EDR implementation in 13 Bluetooth SoCs. Two vulnerabilities enabling RCE in a BLE stack running on a family of Texas Instruments (TI) chips were published under the name BLEEDINGBIT [54]. Several vulnerabilities called “BleedingTooth” [55] were found in the Linux BlueZ BLE implementation, enabling “zero click” RCE via advertising packets.

5. Summary and perspective

Considering all the vulnerabilities and attacks described in the previous section, it may seem very hard to design a secure BLE device. However, as we have also seen, the Bluetooth SIG responded to most threats and provided ways to mitigate them. Therefore, building a secure BLE device can be achieved for the most part by making use of all security mechanisms provided by the specification in its most current version:

1. Devices should perform bonding in order to establish a shared LTK. The LTK can then be used to secure all future sessions.
2. Legacy pairing (except OOB authentication), just-works or using a fixed passkey is insecure and must be avoided. To eliminate the Fixed Coordinate and KNOB vulnerabilities, an implementation following at least version 5.1 should be used.
3. Devices should use secure connection only (SCO) mode introduced with version 4.2 (see Section 3.3) which implies an authenticated LE-SC association method and the use of the Diffie-Hellman key establishment protocol. With beginning of Bluetooth version 5.0 it also enforces the use of a 128 bit encryption key size for the AES-CCM encryption and authentication. One pitfall here is that current mobile and desktop operating systems provide little means of enforcing SCO mode on the GATT client side (Downgrade attacks described in 4.7). Applications should set and check security settings as tightly as possible and cancel a connection if any error or deviation from the expected is observed.

4. With passkey entry and numeric comparison there is an additional risk of user errors and implementation flaws enabling attacks such as visually eavesdropping the passkey or the Pairing Method Confusion Attack. Depending on the device’s capabilities, developing an own out-of-band (OOB) pairing method can also be an option. For instance, a temporary cable connection would provide a channel for a key exchange that is very hard to eavesdrop or manipulate.
 5. Care must be taken with devices supporting both BLE and BR/EDR as the CTKD feature may allow the LTK to be overwritten through a BLURtooth attack under certain circumstances (see Section 4.6).
 6. In order to protect the device user’s privacy, BLE’s privacy feature should be enabled in order to use randomized private addresses instead of trackable identity addresses. To prevent tracking of private addresses themselves, these need to be renewed frequently. While the specification suggests a time-span of 15 minutes for address use, it is better to adjust the time-span to the communication pattern and threat scenario specific to the device and to balance it with other constraints such as the overhead caused by address renewal. Moreover, the device must not leak information that could be used for fingerprinting and tracking in any other way. One such example would be to make the serial number available through advertisements or public characteristics.
- In addition to tracking or localization of an individual device, it may also be problematic if third parties can infer from BLE traffic that a user has a particular type of device. For instance, the use of a medical device with BLE interface may allow conclusions about a user’s illness. Therefore, the potential leakage of such information, e.g. through the public device name characteristic should be considered as privacy relevant, too.
7. Particularly for BLE devices used in conjunction with an app on a mobile OS, it may be considered to complement BLE security with an own application layer security protocol. One benefit of this is that the device can be associated with the specific app. This eliminates co-located app attacks.

Fixing known security weaknesses in devices that are already in the field usually requires patching or updating the vulnerable code. Especially for devices that are likely to remain in use over many years, the capability to update at least all software layers of the stack is an important feature to mitigate potential future security issues. Bluetooth modules that do not allow a complete firmware update still may have a small RAM area that allows the software stack to apply a limited number of patches at run-time.

A broader set of recommendations including security management and covering both BLE and BR/EDR (only up-to version 4.2 though) was prepared by the NIST [56].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Appropriate Language Mapping Tables, Tech. rep., Bluetooth SIG, 2021, URL https://specificationrefs.bluetooth.com/language-mapping/Appropriate_Language_Mapping_Table.pdf.
- [2] Bluetooth Core Specification Revision 5.2, Bluetooth SIG, 2019, URL https://www.bluetooth.org/docman/handlers/downloadaddoc.aspx?doc_id=478726.
- [3] Bluetooth SIG, Bluetooth SIG specification list, withdrawal of version 4.0, 2021, URL <https://www.bluetooth.com/specifications/specs/?status=all&keyword=Core+Specification+4.0&filter=>.
- [4] D. Whiting, R. Housley, N. Ferguson, Counter with CBC-MAC (CCM), RFC 3610, IETF, 2003, URL <https://tools.ietf.org/html/rfc3610>.

- [5] Cryptographic Mechanisms: Recommendations and Key Lengths, Part 1, BSI Technical Guideline TR-02102-1, Bundesamt für Sicherheit in der Informationstechnik (BSI), 2020, URL https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile&v=10.
- [6] N.P. Smart (Ed.), ECRYPT-CSA: Algorithms, Key Size and Protocols Report (2018), Tech. rep., University of Bristol, 2018, URL <https://www.ecrypt.eu.org/csa/documents/D5.4-FinalAlgKeySizeProt.pdf>.
- [7] E. Barker (Ed.), Recommendation for Key Management: Part 1 – General, Tech. rep. SP.800-57, National Institute of Standards and Technology, 2020, <http://dx.doi.org/10.6028/NIST.SP.800-57pt1r5>.
- [8] M. Ryan, Bluetooth: with low energy comes low security, in: 7th USENIX Workshop On Offensive Technologies, 2013, URL <https://www.usenix.org/conference/woot13/workshop-program/presentation/ryan>.
- [9] D. Cauquil, You'd better secure your BLE devices or we'll kick your butts!, DEF CON 26, 2018, <https://media.defcon.org/DEF%20CON%2026/DEF%20CON%2026%20presentations/DEFCON-26-Damien-Cauquil-Secure-Your-BLE-Devices-Updated.pdf>.
- [10] S. Jasek, GATTacking Bluetooth Smart Devices, Tech. rep., SecuRing, p. 15, URL <http://gattack.io/whitepaper.pdf>.
- [11] D. Cauquil, BtleJuice framework, URL <https://github.com/DigitalSecurity/btlejuice>.
- [12] R. Cayre, F. Galtier, G. Auriol, V. Nicomette, M. Kaënche, G. Marconato, InjectaBLE: INjecting malicious traffic into established Bluetooth Low Energy connections, in: IEEE/IFIP International Conference On Dependable Systems And Networks (DSN), Taipei (virtual), Taiwan, 2021, URL <https://hal.laas.fr/hal-03193297>.
- [13] T. Issoufaly, P.U. Tournoux, BLEB: Bluetooth low energy botnet for large scale individual tracking, in: 2017 1st International Conference On Next Generation Computing Applications (NextComp), 2017, pp. 115–120, <http://dx.doi.org/10.1109/NEXTCOMP.2017.8016185>.
- [14] A.K. Das, P.H. Pathak, C.-N. Chuah, P. Mohapatra, Uncovering privacy leakage in BLE network traffic of wearable fitness trackers, in: Proceedings Of The 17th International Workshop On Mobile Computing Systems And Applications, in: HotMobile '16, Association for Computing Machinery, 2016, pp. 99–104, <http://dx.doi.org/10.1145/2873587.2873594>.
- [15] J.K. Becker, D. Li, D. Starobinski, Tracking anonymized Bluetooth devices, Proc. Priv. Enhancing Technol. 2019 (3) (2019) 50–65, <http://dx.doi.org/10.2478/popets-2019-0036>, URL <https://content.sciendo.com/view/journals/popets/2019/3/article-p50.xml>.
- [16] G. Celosia, M. Cunche, Saving private addresses: an analysis of privacy issues in the Bluetooth-low-energy advertising mechanism, in: Proceedings Of The 16th EAI International Conference On Mobile And Ubiquitous Systems: Computing, Networking And Services, ACM, 2019, pp. 444–453, <http://dx.doi.org/10.1145/3360774.3360777>, URL <https://hal.inria.fr/hal-02394629>.
- [17] G. Celosia, M. Cunche, Discontinued privacy: Personal data leaks in apple Bluetooth-Low-Energy continuity protocols, Proc. Priv. Enhancing Technol. 2020 (1) (2020) 26–46, <http://dx.doi.org/10.2478/popets-2020-0003>, URL <https://www.sciendo.com/article/10.2478/popets-2020-0003>.
- [18] G. Celosia, M. Cunche, Fingerprinting Bluetooth-Low-Energy devices based on the generic attribute profile, in: Proceedings Of The 2nd International ACM Workshop On Security And Privacy For The Internet-Of-Things - IoT S&P'19, ACM Press, 2019, pp. 24–31, <http://dx.doi.org/10.1145/3338507.3358617>, URL <http://dl.acm.org/citation.cfm?doi=10.1145/3338507.3358617>.
- [19] J. Martin, D. Alpuche, K. Bodeman, L. Brown, E. Fenske, L. Foppe, T. Mayberry, E.C. Rye, B. Sipes, S. Teplov, Handoff all your privacy - A review of apple's Bluetooth Low Energy continuity protocol, Proc. Priv. Enhancing Technol. 2019 (4) (2019) 34–53, <http://dx.doi.org/10.2478/popets-2019-0057>.
- [20] A. Korolova, V. Sharma, Cross-app tracking via nearby Bluetooth Low Energy devices, in: Proceedings Of The Eighth ACM Conference On Data And Application Security And Privacy, in: CODASPY '18, Association for Computing Machinery, 2018, pp. 43–52, <http://dx.doi.org/10.1145/3176258.3176313>.
- [21] M. Caesar, J. Steffan, A location privacy analysis of Bluetooth mesh, J. Inf. Secur. Appl. 54 (2020) 102563, <http://dx.doi.org/10.1016/j.jisa.2020.102563>, URL <https://linkinghub.elsevier.com/retrieve/pii/S2214212619310506>.
- [22] P.-O. Delhaye, J. Reardon, SwissCovid: a critical analysis of risk assessment by Swiss authorities, 2020, arXiv:2006.10719, URL <http://arxiv.org/abs/2006.10719>.
- [23] D. Dolev, A.C. Yao, On the security of public key protocols, IEEE Trans. Inf. Theory IT-29 (2) (1983) URL <https://www.cs.huji.ac.il/~dolev/pubs/dolev-yao-ieee-01056650.pdf>.
- [24] T. Rosa, Bypassing Passkey Authentication in Bluetooth Low Energy, Report 2013/309, Cryptology ePrint Archive, 2013, URL <https://eprint.iacr.org/2013/309>.
- [25] T. Claverie, J. Lopes-Esteves, Testing for weak key management in Bluetooth Low Energy implementations, in: Symposium Sur La Sécurité Des Technologies De L'Information et des Communications, SSTIC 2021, Elle a eu lieu à Rennes, 2020, p. 38, URL https://www.sstic.org/2020/presentation/testing_for_weak_key_management_in_bluetooth_low_energy_implementations/.
- [26] E. Biham, L. Neumann, Breaking the Bluetooth Pairing – Fixed Coordinate Invalid Curve Attack, Tech. rep., Technion's Hiroshi Fujiwara Cyber Security Research Center & Computer Science Department Technion – Israel Institute of Technology, 2018, URL <http://www.cs.technion.ac.il/~biham/BT/>.
- [27] Validation of Elliptic Curve Parameters, Bluetooth SIG, 2018, URL <https://www.bluetooth.com/learn-about-bluetooth/bluetooth-technology/bluetooth-security/bluetooth-sig-security-update/>.
- [28] C. Cremers, D. Jackson, Prime, order please! revisiting small subgroup and invalid curve attacks on protocols using Diffie-Hellman, in: 2019 IEEE 32nd Computer Security Foundations Symposium (CSF), 2019, pp. 78–7815, <http://dx.doi.org/10.1109/CSF.2019.00013>.
- [29] Pairing Updates, Erratum 10734, Bluetooth SIG, 2018, URL https://www.bluetooth.org/docman/handlers/downloadaddoc.aspx?doc_id=447440.
- [30] D. Antonioli, N.O. Tippenhauer, K.B. Rasmussen, The KNOB is broken: Exploiting low entropy in the encryption key negotiation of bluetooth BR/EDR, in: 28th USENIX Security Symposium (USENIX Security 19), USENIX Association, 2019, pp. 1047–1061, URL <https://www.usenix.org/conference/usenixsecurity19/presentation/antonioli>.
- [31] D. Antonioli, N.O. Tippenhauer, K. Rasmussen, Low Entropy Key Negotiation Attacks on Bluetooth and Bluetooth Low Energy, Tech. rep. Report 2019/933, Cryptology ePrint Archive, 2019, URL <https://eprint.iacr.org/2019/933>.
- [32] D. Antonioli, N.O. Tippenhauer, K. Rasmussen, Key negotiation downgrade attacks on Bluetooth and Bluetooth Low Energy, ACM Trans. Priv. Secur. 23 (3) (2020) 14:1–14:28, <http://dx.doi.org/10.1145/3394497>, URL <https://francozappa.github.io/publication/knob-ble/paper.pdf>.
- [33] Key Negotiation of Bluetooth, Bluetooth SIG, 2019, URL <https://www.bluetooth.com/learn-about-bluetooth/bluetooth-technology/bluetooth-security/statement-key-negotiation-of-bluetooth/>.
- [34] Encryption Key Size Updates, Expedited Errata Correction 11838, Bluetooth SIG, 2019, URL https://www.bluetooth.org/docman/handlers/DownloadDoc.aspx?doc_id=470741.
- [35] M. von Tschirschnitz, L. Peuckert, F. Franzen, J. Grossklags, Method confusion attack on Bluetooth pairing, in: 2021 IEEE Symposium On Security And Privacy (SP), IEEE Computer Society, 2021, pp. 213–228, <http://dx.doi.org/10.1109/SP40001.2021.00013>, URL <https://www.computer.org/csdl/pds/api/csdl/proceedings/download-article/1mbmHzm2Q6c/pdf>.
- [36] Bluetooth SIG Statement Regarding the Method-Confusion Pairing Vulnerability, Bluetooth SIG, 2020, URL <https://www.bluetooth.com/learn-about-bluetooth/bluetooth-technology/bluetooth-security/method-vulnerability/>.
- [37] A.Y. Lindell, Attacks on the pairing protocol of Bluetooth V2, in: BlackHat USA, 2008, p. 10, URL https://docs.huihoo.com/blackhat/usa-2008/BH_US_08_Lindell_Bluetooth_2.1_New_Vulnerabilities.pdf.
- [38] J. Barnickel, J. Wang, U. Meyer, Implementing an attack on Bluetooth 2.1+ Secure Simple Pairing in passkey entry mode, in: 2012 IEEE 11th International Conference On Trust, Security And Privacy In Computing And Communications, 2012, pp. 17–24, <http://dx.doi.org/10.1109/TrustCom.2012.182>.
- [39] D.-Z. Sun, Y. Mu, W. Susilo, Man-in-the-middle attacks on Secure Simple Pairing in Bluetooth standard V5.0 and its countermeasure, Personal Ubiquitous Comput. 22 (1) (2018) 55–67, <http://dx.doi.org/10.1007/s00779-017-1081-6>, URL <https://ro.uow.edu.au/cgi/viewcontent.cgi?article=1706&context=eisapers1>.
- [40] R. Östádal, Evaluation of Bluetooth Security, Tech. rep., 2011, p. 10, URL https://is.muni.cz/www/255508/bluetooth_security.pdf.
- [41] T. Claverie, J.L. Esteves, BlueMirror: Reflections on Bluetooth pairing and provisioning protocols, in: 2021 IEEE Security And Privacy Workshops SPW, 2021, pp. 339–351, <http://dx.doi.org/10.1109/SPW53761.2021.00054>.
- [42] Bluetooth SIG Statement Regarding the 'Authentication of the LE Legacy Pairing' Vulnerability, Bluetooth SIG, 2021, URL <https://www.bluetooth.com/learn-about-bluetooth/key-attributes/bluetooth-security/legacy-pairing/>.
- [43] Bluetooth SIG Statement Regarding the 'Impersonation in the Passkey Entry Protocol' Vulnerability, Bluetooth SIG, 2021, URL <https://www.bluetooth.com/learn-about-bluetooth/key-attributes/bluetooth-security/passkey-entry/>.
- [44] D. Antonioli, N.O. Tippenhauer, K. Rasmussen, M. Payer, BLURtooth: Exploiting cross-transport key derivation in Bluetooth Classic and Bluetooth Low Energy, 2020, Arxiv, arXiv:2009.11776, URL <http://arxiv.org/abs/2009.11776>.
- [45] Bluetooth SIG Statement Regarding the Exploiting Cross-Transport Key Derivation in Bluetooth Classic and Bluetooth Low Energy (BLURtooth) and the Security implications of key conversion between BR/EDR and BLE Vulnerabilities, Bluetooth SIG, 2020, URL <https://www.bluetooth.com/learn-about-bluetooth/bluetooth-technology/bluetooth-security/blurtooth/>.
- [46] J. Wu, Y. Nan, V. Kumar, D.J. Tian, A. Bianchi, M. Payer, D. Xu, BLES: Spoofing attacks against reconnections in Bluetooth Low Energy, in: 14th USENIX Workshop On Offensive Technologies (WOOT 20), USENIX Association, 2020, URL <https://www.usenix.org/conference/woot20/presentation/wu>.
- [47] Y. Zhang, J. Weng, R. Dey, Y. Jin, Z. Lin, X. Fu, On the (in)security of Bluetooth Low Energy one-way secure connections only mode, 2019, Arxiv, arXiv:1908.10497, URL <http://arxiv.org/abs/1908.10497>.
- [48] Y. Zhang, J. Weng, R. Dey, Y. Jin, Z. Lin, X. Fu, Breaking secure pairing of Bluetooth Low Energy using downgrade attacks, in: 29th USENIX Security Symposium USENIX Security 20, 2020, pp. 37–54, URL <https://www.usenix.org/conference/usenixsecurity20/presentation/zhang-yue>.

- [49] P. Sivakumaran, J. Blasco, A study of the feasibility of co-located app attacks against BLE and a large-scale analysis of the current application-layer security landscape, in: 28th USENIX Security Symposium 19, 2019, pp. 1–18, URL <https://www.usenix.org/conference/usenixsecurity19/presentation/sivakumaran>.
- [50] J. Wang, F. Hu, Y. Zhou, Y. Liu, H. Zhang, Z. Liu, BlueDoor: Breaking the secure information flow via BLE vulnerability, in: Proceedings Of The 18th International Conference On Mobile Systems, Applications, And Services, in: MobiSys '20, Association for Computing Machinery, 2020, pp. 286–298, <http://dx.doi.org/10.1145/3386901.3389025>, URL <http://tns.thss.tsinghua.edu.cn/~jiliang/publications/MOBISYS2020.BlueDoor.pdf>.
- [51] M.E. Garbelini, C. Wang, S. Chattopadhyay, S. Sumei, E. Kurniawan, SweynTooth: Unleashing mayhem over Bluetooth Low Energy, in: 2020 USENIX Annual Technical Conference (USENIX ATC 20), 2020, pp. 911–925, URL <https://www.usenix.org/conference/atc20/presentation/garbelini>.
- [52] B. Seri, G. Vishnepolsky, BlueBorne, The Dangers of Bluetooth Implementations: Unveiling Zero Day Vulnerabilities and Security Flaws in Modern Bluetooth Stacks, ARMIS inc., 2017, URL <http://go.armis.com/blueborne-technical-paper>.
- [53] M.E. Garbelini, S. Chattopadhyay, V. Bedi, S. Sun, E. Kurniawan, BrakTooth: Causing Havoc on Bluetooth Link Manager, Vulnerability disclosure report, ASSET Research Group, Singapore University of Technology and Design, 2021, p. 17, URL <https://asset-group.github.io/disclosures/braktooth/braktooth.pdf>.
- [54] B. Seri, G. Vishnepolsky, D. Zusman, BLEEDINGBIT: The Hidden Attack Surface within BLE Chips, Technical white paper, ARMIS inc., 2018, URL <https://www.armis.com/bleedingbit/>.
- [55] A. Nguyen, BleedingTooth: Linux Bluetooth zero-click remote code execution, URL <https://google.github.io/security-research/pocs/linux/bleedingtooth/writeup.html>.
- [56] J. Padgett, et al., Guide to Bluetooth Security, Revision 2, Special Publication (NIST SP) 800–121, National Institute of Standards and Technology, 2017, URL DOI: 10.6028/NIST.SP.800-121r2.



Matthias Cäsar works as a research associate at the Fraunhofer Institute for Secure Information Technology (SIT) in Darmstadt, Germany. His main research topics are security and privacy aspects of IoT protocols, in particular Bluetooth Low Energy and Bluetooth Mesh. As a member of the Security Analyses and Tests group, he contributes to projects with industrial partners in the field of IoT security. Matthias Cäsar studied electrical engineering with focus on information technology at the RheinMain University of Applied Sciences.



Tobias Pawelke works as a research associate at the Fraunhofer Institute for Secure Information Technology (SIT) in Darmstadt, Germany. There he is working at the “Security Analysis and Tests” workgroup and researches in the field of fuzz testing wireless protocols on embedded systems. Prior to his current profession, he worked two years as a cyber security consultant and supported customers in the field of security mechanisms in mobile operating systems and secure distance bounding in automotive systems. Currently Tobias Pawelke studies at the Technical University of Darmstadt pursuing his master degree in IT-Security.



Jan Steffan leads the Security Analyses and Tests group at the Fraunhofer Institute for Secure Information Technology (SIT) in Darmstadt, Germany.

After he received his diploma of computer science in 1999, Jan Steffan worked within an applied research group at Darmstadt University of Technology. He acquired a broad background on IT technologies and IT security by conducting a range of IT security related R&D projects with partners from industry. He joined the IT-Security Testlab at Fraunhofer SIT in 2005 as a researcher, project manager and a core member of the testing team. His main area of work is the evaluation and verification of IT security properties of complex IT systems, hardware and software products. His current research topic is the development of new and innovative security testing methodologies for embedded devices.



Gabriel Terhorst works as a research associate at the Fraunhofer Institute for Secure Information Technology (SIT) in Darmstadt, Germany. His research is focused mainly on the security and privacy aspects of wireless protocols used in smart homes and the IoT. As a member of the Security Analysis and Tests group, he takes part in security audits during the design phase of upcoming products. Gabriel Terhorst studied Computer Science with a focus on IT-Security at the University of Applied Sciences in Darmstadt.